

Interview Preparation Guide

Sachin Himmat Koli — Application Support Engineer, Accenture (Bengaluru)

Contents

HOW TO USE THIS GUIDE

PART 1 — RESUME DEEP-DIVE ANALYSIS

- 1.1 Your profile in one line
- 1.2 Where you are genuinely strong
- 1.3 Where you are thin (be honest with yourself here)
- 1.4 THE FOUR DANGER QUESTIONS
- 1.5 CROSS-QUESTION MAP
- 1.6 THREE THINGS TO DO BEFORE YOU APPLY

PART 2 — YOUR CORE STORY

- 2.1 Tell me about yourself
- 2.2 Walk me through your resume
- 2.3 Why are you looking for a job change?
- 2.4 Why do you want to join Accenture?
- 2.5 Why should we hire you?
- 2.6 Where do you see yourself in five years?

PART 3 — RESUME-BASED QUESTIONS (Q1–Q20)

PART 4 — ROLE-BASED QUESTIONS (Q21–Q32)

PART 5 — PROJECT-BASED QUESTIONS (Q33–Q44)

PART 6 — TECHNICAL QUESTIONS (Q45–Q92)

- 6.1 BEGINNER LEVEL (Q45–Q57)
- 6.2 INTERMEDIATE LEVEL (Q58–Q74)
- 6.3 ADVANCED LEVEL (Q75–Q84)
- 6.4 TROUBLESHOOTING SCENARIOS (Q85–Q92)

PART 7 — BEHAVIOURAL QUESTIONS (Q93–Q108)

PART 8 — HR ROUND (Q109–Q131)

PART 9 — MANAGERIAL ROUND (Q132–Q152)

PART 10 — CLIENT ROUND (Q153–Q173)

PART 11 — DIFFICULT AND TRICKY QUESTIONS (Q174–Q188)

PART 12 — PRIORITY LISTS

- 12.1 TOP 30 MOST IMPORTANT QUESTIONS
- 12.2 TOP 20 RESUME-BASED QUESTIONS
- 12.3 TOP 20 HR QUESTIONS
- 12.4 TOP 20 TECHNICAL QUESTIONS
- 12.5 TOP 20 PROJECT-BASED QUESTIONS

PART 13 — FINAL CHECKLIST

- Before you apply
 - Your three STAR stories (write them out and rehearse)
 - The four danger answers (rehearse out loud until natural)
- On the day
 - Three habits that will carry you

HOW TO USE THIS GUIDE

Do not read this end to end the night before. Work it in this order:

1. **Part 1 (Deep-Dive)** — read this first and twice. It tells you where you are strong, where you are thin, and the four questions that can sink you.
2. **Part 2 (Core Story)** — memorise the *structure*, not the words. These six answers get reused in every round.
3. **Part 5 (Technical)** — this is where a 1-year candidate usually loses. Give it the most hours.
4. **Parts 6–8 (HR, Managerial, Client)** — read once, practise out loud twice.
5. **Part 10 (Priority Lists)** — your final 48-hour revision plan.

Say every answer out loud at least once. An answer that reads well often sounds rehearsed when spoken. If it sounds like a script, cut it in half.

PART 1 — RESUME DEEP-DIVE ANALYSIS

1.1 Your profile in one line

L2 application support analyst at Cognizant, Pune, 13 months' experience on ServiceNow ITSM and CMDB for a large enterprise client, monitoring five enterprise platforms, ServiceNow Administration Fundamentals certified, CSA in progress, BCA with 9.52 CGPA, MCA in progress.

1.2 Where you are genuinely strong

You are L2, not L1. Most applicants for this role are L1 ticket routers. You receive escalations that someone else already failed to solve. Lead with this in every round. It is your single biggest differentiator.

You have platform breadth, not just ticket breadth. AppDynamics, Control-M, Informatica, Salesforce and Azure. The JD calls the role "software detectives." Monitoring and batch job failures are detective work. Very few 1-year candidates can talk about a failed Control-M job.

You can query a database. SQL and PL/SQL on Oracle. A support engineer who can validate data instead of forwarding the ticket is worth noticeably more.

CMDB ownership. Configuration item accuracy is unglamorous and most juniors have never touched it. It shows you were trusted with data integrity, not just ticket closure.

Academics are strong. 9.52 CGPA in BCA. Accenture weighs academics in early-career hiring more than most companies.

Certification is current. Administration Fundamentals completed Aug 2026, one month ago. That is fresh and shows initiative rather than a certificate collected years back.

You have Bengaluru history. Trained and interned there on-site. This kills the relocation doubt before it is raised.

1.3 Where you are thin (be honest with yourself here)

Gap	Why it matters	What to do
13 months' total experience	You will be judged as a fresher-plus	Lean on depth of what you did, not duration
No ITIL certification	JD asks for incident management best practices	Say "ITIL-aligned process, certification in progress" — never imply you hold it
Voice support is limited	JD lists it as good-to-have	Use your real user calls; do not inflate them
Five tools listed, shallow depth likely	Classic cross-question trap	Pre-decide which two you can go deep on
No ServiceNow development work	Your PDI practice is thin	Build one small thing before the interview
Single client, single account	Limited exposure variety	Frame it as depth on a critical system
Achievement is from a 2-month 2023 internship	Reads thin for a 2026 interview	Build a work achievement to replace it
Notice period not decided	HR will ask in round one	Get the exact number from your HR before you apply

1.4 THE FOUR DANGER QUESTIONS

These four can cost you the offer. Prepare these before anything else in this guide.

DANGER 1 — "You have only been at Cognizant 13 months. Why are you leaving already?"

This is the hardest question you will face and it will come up in HR, managerial and client rounds.

Why it is dangerous: Short tenure signals flight risk. Accenture spends money on onboarding and background verification. If they think you will leave in a year, they will pick someone else even if you are technically better.

What NOT to say: No growth. Bad manager. Boring work. Better salary. Night shifts are hard. All of these tell them you will say the same about Accenture in a year.

What to say instead — anchor on scope, not dissatisfaction:

"I am not unhappy at Cognizant. My first year there gave me the ITSM foundation I needed, and I got L2 ownership faster than I expected. What I want next is more depth on the application side. Right now a large part of my day is process and CMDB work. This role is described as troubleshooting across multiple components of critical business systems, which is the part of my job I am best at and want to do more of. Bengaluru is also where most of the platform work sits, and I have already worked there on-site, so it is not a new city for me."

Then close the loop yourself, because they are worried about it and will not say so:

"On tenure, I understand the concern. I am not looking for a short move. I am mid-way through my CSA and my MCA, and both point at building a long career in this platform. I want the next several years in one place where I can go deep."

DANGER 2 — "Your other resume says Aspiring ServiceNow Developer. Do you actually want a support role?"

Why it is dangerous: This is the one that quietly loses offers. If the panel concludes you want to be a developer and are treating support as a stepping stone, they will not hire you. They have seen people leave for a dev role after eight months.

They may not have your other resume, but this will surface anyway. Your resume lists JavaScript, Flask, PHP and full-stack projects, and your CSA is a platform certification. A sharp interviewer will connect the dots and ask.

Do not lie. If you deny any interest in development and they later see your GitHub or LinkedIn, you look dishonest.

Honest framing that still gets you hired:

"I do want to grow deep on the ServiceNow platform, and administration and support are where that starts. My view is that the people who write good platform solutions are the ones who have supported them in production first. I have seen changes go live that looked fine in design and then generated thirty tickets, because nobody had handled the incidents. That is the experience I am building now. Application support is not a detour for me, it is the foundation, and the CSA I am doing is an administration certification, not a developer one."

Key move: frame development knowledge as making you a *better* support engineer, not as your exit plan.

DANGER 3 — "You list AppDynamics, Control-M, Informatica, Salesforce and Azure. Take me through each."

Why it is dangerous: Five tools on a 13-month resume invites a depth test. If you fumble three of them, the panel discounts the whole skills section, including the parts you are actually good at.

Prepare now: Pick the **two** you genuinely know. Most likely AppDynamics and Control-M. For those two, be ready with: what you look at, one specific alert or failure you handled, and what you did next.

For the other three, answer honestly and narrowly:

"My depth is uneven across those and I would rather be straight about it. On AppDynamics and Control-M I work daily and can talk through real failures. On Informatica my involvement is monitoring load failures and raising them with the data team rather than fixing mappings. Salesforce and Azure are platforms where I handle access and application issues raised by users, not administration."

This answer scores well. Panels trust a candidate who marks their own boundaries. Overclaiming and then collapsing under follow-ups is far worse.

DANGER 4 — "You are doing an MCA from 2025 to 2027 while working full time. How?"

Why it matters: Accenture runs background verification on education. An unexplained overlap between full-time employment and a full-time degree can delay or void an offer. They also worry that classes will clash with rotational shifts.

What to do:

- Know the exact mode of your MCA — distance, online or part-time — and say it plainly.
- Confirm your BCA gives you the 15 years of full-time education on its own (10 + 2 + 3). The MCA is extra, not load-bearing.
- Reassure on shifts.

"The MCA is a [distance/part-time] programme, so it runs alongside work and does not clash with shifts. My eligibility for this role comes from my BCA, which is 15 years of full-time education on its own. The MCA is something I am doing on top, and I have already managed it for over a year including shift work."

1.5 CROSS-QUESTION MAP

For each resume line, the questions a panel will drill into. Prepare an answer for every row.

"Handle around 10 tickets a day, 95% SLA compliance"

- Why only 10? What is the ticket mix?
- What counts as SLA breach on your account? Response or resolution?
- What happens when you are about to breach?
- Give me the last ticket you closed.
- What is your reopen rate?

"Take escalations from L1, move unresolved P2 to L3"

- What makes a ticket L2 rather than L1?
- What do you include when you escalate to L3?
- Give me an example of something you sent to L3. What had you already ruled out?
- Have you ever pushed a ticket back to L1? Why?
- How is P2 defined on your account?

"Keep CMDB configuration items and relationships accurate"

- What CI classes do you work with?
- How do CIs get into your CMDB — Discovery, import, manual?
- What is a CI relationship and why does it matter for change?
- What do you do when you find a duplicate CI?
- What is the reconciliation or identification rule concept?

"Monitor AppDynamics, Control-M, Informatica"

- What do you check first in AppDynamics when an app slows down?
- A Control-M job fails at 2am. Walk me through your next ten minutes.
- What is the difference between a job failure and a job that is stuck?
- Who do you call and when?

"Check application logs and run SQL queries on Oracle"

- Which log files, and what do you grep for?
- Write a query that finds duplicate records in a table.
- What is the difference between a JOIN and a subquery?
- Have you ever run an UPDATE in production? Under what approval?

"ServiceNow: UI Policies, Data Policies, Update Sets, ACLs"

- Difference between a UI Policy and a Client Script?
- Difference between a UI Policy and a Data Policy?
- What is an Update Set and what does it not capture?
- What are the four things an ACL checks?
- Difference between a Business Rule and a Client Script?

"Personal Developer Instance practice"

- What have you actually built on it?
- Show me or describe it end to end.
- **Warning:** if you have not built anything real, remove this line from your resume before you apply.

"Excel macros and VBA for ticket and SLA reporting"

- What does your report show?
- Did you write the macro or use one that existed?
- Why Excel and not a ServiceNow dashboard?

"Knowledge base articles for recurring issues"

- How many have you written?
- Give me the topic of one.
- How do you know it reduced escalations?

"Ranked first in machine test at ASORG Softtech"

- What was the test?
- That was 2023 and a 2-month internship. What have you achieved at Cognizant?
- **Action:** build a Cognizant achievement to replace this. See section 1.6.

"Smart Summarize / Real Estate Management System" (if you re-add projects)

- Which part did you write yourself?
- What was the hardest bug?
- How did you handle API limits and errors?

1.6 THREE THINGS TO DO BEFORE YOU APPLY

1. Get one real achievement from Cognizant. Right now your only listed achievement is from a 2-month internship in 2023. Find one of these from the last year and quantify it: a recurring incident you got permanently fixed, a KB article that cut L1 escalations, a CMDB clean-up you did, a report you built that the team still uses, a shift handover process you improved.

2. Build one thing on your PDI. Two evenings is enough. Create a catalog item with a two-stage approval, one Business Rule, one UI Policy and one Flow. Then you can answer "what have you built" with something real. If you will not do this, delete the PDI line from your resume.

3. Get your exact notice period in writing from Cognizant HR. Every single round will ask.

PART 2 — YOUR CORE STORY

These six answers get reused constantly. Learn the structure, not the wording.

2.1 Tell me about yourself

Simple answer:

"I am Sachin Koli. I work as an Analyst at Cognizant in Pune, doing L2 application support on the ServiceNow ITSM platform for a large enterprise client. I handle incidents, problems, changes and requests, keep the CMDB accurate, and monitor tools like AppDynamics and Control-M to keep production stable. I have a BCA with 9.52 CGPA, I am ServiceNow Administration Fundamentals certified, and I am working on my CSA. I am looking for an application support role where I can go deeper into troubleshooting critical business systems."

Detailed professional answer:

"I am Sachin Koli, currently an Analyst at Cognizant in Pune, and I have been there since August 2025 after completing my on-site internship and training in Bengaluru.

My work is L2 application support on ServiceNow ITSM for a large enterprise client. Tickets reach me after L1 has triaged them, so most of my day is diagnosis rather than routing. I handle around ten incidents, problems, changes and requests a day against a 95 percent SLA target, and where I cannot resolve something I escalate to L3 with proper root cause notes.

Alongside incident work I own CMDB accuracy, which means keeping configuration items and their relationships correct so change impact is assessed properly. I also monitor AppDynamics for application performance, Control-M for batch jobs and Informatica for data loads, and I use SQL on Oracle to check application data when users report mismatches.

On qualifications, I have a BCA with 9.52 CGPA and I am doing my MCA alongside work. I completed ServiceNow Administration Fundamentals last month and my CSA is scheduled for November.

I am looking at this role because it is described as troubleshooting across multiple components of critical business systems, which is the strongest and most enjoyable part of my current job, and I would like to do more of it."

Tips: Keep it 60 to 90 seconds. Present role first, education last, and finish by pointing at the role you are interviewing for. Mention the L2 tier early, because it reframes how they hear everything after it.

Common mistakes: Starting from your school. Reciting your resume line by line. Running past two minutes. Forgetting to end with why you are sitting there.

Follow-ups: What does a typical day look like? Which part of that do you enjoy most? Why support rather than development?

2.2 Walk me through your resume

Simple answer:

"I finished my BCA in June 2025 with 9.52 CGPA. Before that, in late 2023, I did a short full-stack internship at ASORG Softtech where I built responsive websites and came first in their machine test. In June 2025 I joined Cognizant as an intern in Bengaluru, on-site, where I was trained in ITSM, ServiceNow, SQL, PL/SQL, Oracle and Linux. In August 2025 I moved into the operations team in Pune as an Analyst, doing L2 application support and CMDB work, which is what I do now."

Detailed professional answer: Same sequence, but at each stage say what changed in your capability:

"Chronologically: my BCA gave me the programming base, mainly SQL, Python and JavaScript, and I finished with 9.52 CGPA. The ASORG internship in 2023 was short, two months, and it was web development rather than support, but it is where I first worked in a team of four to a delivery schedule.

The Cognizant internship in mid-2025 was the real turn. It was on-site in Bengaluru and covered ITSM process, ServiceNow, SQL and PL/SQL, Oracle and Linux. That is where support went from being a job title to something I understood as a process.

From August 2025 I have been an Analyst in Pune on L2 application support and CMDB. In the first few months I was mostly learning the account and shadowing. Over the last year I have taken ownership of CMDB accuracy for my scope, moved into handling L1 escalations myself, and picked up the monitoring rotation across AppDynamics, Control-M and Informatica.

In parallel I have kept studying: MCA alongside work, and ServiceNow Administration Fundamentals completed last month with CSA next."

Tips: Go oldest to newest. Show progression at each step, because with 13 months you must show a curve rather than a flat line.

Common mistakes: Reading the resume aloud. Spending equal time on a 2-month internship and your current role. Skipping the progression story.

Follow-ups: Why the shift from full-stack to support? What was the biggest thing you learned in training? What has changed in your role since you joined?

2.3 Why are you looking for a job change?

See **Danger 1** in Part 1. That is the full answer. Learn it properly.

2.4 Why do you want to join Accenture?

Simple answer:

"Two reasons. First, the role itself. The description talks about identifying and solving issues across multiple components of critical business systems, and that troubleshooting side is what I am strongest at. Second, Accenture's scale. Working across different clients and platforms would broaden me, because right now I have depth on one account but not variety."

Detailed professional answer:

"The role description is what caught me first. It calls the team software detectives and talks about identifying and resolving issues across components of critical business systems. That is very close to what I already do at L2, but the scope sounds wider than a single account."

The second reason is exposure. My year at Cognizant has given me real depth, but on one client and one platform stack. Accenture works across many clients and technologies, and for someone at my stage that variety compounds quickly. I want to see how service desk and application support run in more than one environment.

Third, and practically, Bengaluru. I trained and interned there, so I know the city and I am prepared to relocate rather than just willing on paper.

And the role sits on ServiceNow and ITSM, which is exactly where I am investing my certifications. I am not switching direction, I am going deeper in the same direction."

Tips: Name something specific from the JD, in its own words. Have one Accenture-specific fact ready — check their recent ServiceNow or managed services announcements the week of your interview.

Common mistakes: Generic praise ("Accenture is a great brand"). Saying you want to leave Cognizant rather than join Accenture. Mentioning salary here.

Follow-ups: What do you know about our delivery model? Have you applied elsewhere? What if we place you on a different technology?

2.5 Why should we hire you?

Simple answer:

"Three things. I am already L2, so I am used to tickets that L1 could not solve. I do not just handle tickets, I also monitor the systems behind them, so I can spot a problem before users report it. And I can query databases and read logs, which means I can find causes instead of forwarding them."

Detailed professional answer:

"I would put it in three parts.

First, the tier. I work at L2, so the tickets that reach me have already defeated someone. That is a different skill from routing, and it is what your description of software detectives is asking for.

Second, I work on both sides of the ticket. Most support candidates only know the ticket. I also monitor AppDynamics, Control-M and Informatica, so I have caught failed batch jobs before the business raised anything. Knowing the systems behind the tickets makes root cause faster.

Third, I can investigate technically. SQL and PL/SQL on Oracle, log analysis, and a programming base from my BCA. When a user reports a data mismatch I can go and check the data rather than passing it on.

And I am invested in this direction, not just this job. Administration Fundamentals done, CSA in November, MCA in progress."

Tips: Exactly three points, each tied to a JD line. Close on commitment, because with 13 months they will be weighing your stability.

Common mistakes: Listing personality traits (hardworking, quick learner) with nothing behind them. Claiming things your resume does not support. Comparing yourself to other candidates.

Follow-ups: What would you struggle with in this role? Where are you weakest technically? What if the role is more process than technical?

2.6 Where do you see yourself in five years?

Simple answer:

"I want to be a senior application support or ServiceNow platform person who owns a module end to end. In the next year or two I want to finish my CSA and ITIL and become the person the team escalates to. After that, either leading a support team or moving into platform implementation, depending on where I add more value."

Detailed professional answer:

"Near term, next two years, I want to be genuinely senior on the technical side of support. That means finishing my CSA, adding ITIL Foundation, and becoming the person on the team who takes the hardest tickets and owns problem management for a module rather than just handling incidents as they arrive.

Beyond that I see two paths and I am honestly open to both. One is technical depth, going further into the ServiceNow platform and taking on design and implementation work for the processes I currently support. The other is ownership, running a support team and being accountable for SLA and quality across an account.

What I do not want is to keep moving sideways. I would rather build five years of depth in one place than collect five different job titles."

Tips: That last line directly addresses their tenure worry. Keep the plan inside their organisation.

Common mistakes: "I want to be a manager in 5 years" with no path. Mentioning further study abroad, a startup, or anything that reads as leaving. Saying you want to become a developer, which reopens Danger 2.

Follow-ups: What if there is no team lead opening in two years? Would you move to a different technology if the account needed it? How do you plan to get there?

PART 3 — RESUME-BASED QUESTIONS (Q1–Q20)

Q1. Take me through a typical day in your current role. *(Full treatment — asked almost every time)*

Simple answer:

"I start by checking the monitoring dashboards and overnight alerts, then the shift handover notes. After that I pick up my ticket queue in ServiceNow, work the P2s first, and keep an eye on Control-M jobs through the day. Between tickets I do CMDB updates and any reporting."

Detailed answer:

"First thirty minutes is situational awareness rather than tickets. I check AppDynamics for anything that alerted overnight, look at the Control-M job status for the batch cycle, and read the handover notes from the previous shift so I know what is still open.

Then I go to my ServiceNow queue. I work by priority, so P2s first, then anything close to an SLA target, then the rest. Because I am L2 most of these have already been through L1, so I read what they tried before I start.

Through the day I am also on the monitoring rotation, so if a Control-M job fails or an Informatica load errors, that interrupts the queue and takes priority, because a failed batch job affects everyone rather than one user.

CMDB work happens in gaps and after change windows, updating CIs and relationships where something moved or was decommissioned.

End of shift I write handover notes for anything still open and log where I got to on the P2s."

Tips: Structure it as morning, main block, interrupts, close. Naming a specific tool at each point makes it credible. Mention handover, because 24x7 accounts care about it.

Avoid: "I just work on tickets." Making it sound like you only react and never check anything proactively. Saying you work first-in-first-out with no priority logic.

Follow-ups: What do you do first if a P1 comes in at the same time as an SLA breach? How is your queue assigned — pull or push? How long is your shift and is it rotational?

Q2. Why are you only handling about 10 tickets a day? *(Full treatment — likely follow-up to your resume)*

Simple answer:

"Because they are L2 tickets. They come to me after L1 has already tried, so they need investigation rather than a standard fix. Volume at L2 is lower than L1 by design."

Detailed answer:

"The number reflects the tier and the mix. At L1 on our account the volume is much higher because a lot of it is access requests and known issues with a documented fix. Those get closed in minutes.

What reaches me has already failed L1's attempt, so each one needs log checking, sometimes a database query, and often a conversation with the user to reproduce it. A single data mismatch investigation can take two hours.

On top of that I am on the monitoring rotation and I own CMDB accuracy in my scope, so tickets are not my whole day. If I were doing thirty a day, one of those other things would be getting neglected."

Tips: Turn the low number into evidence of depth. Name the specific non-ticket work that takes up the rest of your time.

Avoid: Sounding defensive or apologetic. Inflating the number on the spot — you would then contradict your own resume.

Follow-ups: What is the longest a ticket has stayed with you? What is your average handling time? What proportion do you close versus escalate?

Q3. What is your SLA target and what happens if you are about to breach it? *(Full treatment)*

Simple answer:

"Our target is 95 percent. If a ticket is getting close, I flag it to my lead before it breaches rather than after, update the ticket with where I have reached, and if it needs L3 I escalate immediately rather than waiting."

Detailed answer:

"We track both response and resolution SLA, and the resolution target on my account is 95 percent, measured monthly.

ServiceNow shows the SLA clock on the ticket, so I know how much time is left. My rule is that I do not let a ticket breach silently. If I can see I will not resolve it in time, I do three things: update the ticket with exactly what I have checked and ruled out, escalate to L3 with those notes if it is beyond my level, and tell my lead so they are not finding out from a report.

Where the delay is on the user's side, waiting for information or approval, I make sure the ticket is in the correct pending state so the clock reflects reality rather than making it look like we are sitting on it.

Genuinely unavoidable breaches happen. What matters is that the reason is documented, because at month end somebody has to explain the number to the client."

Tips: Mentioning pending states shows you understand SLA mechanics rather than just the number. Escalating early is the answer they want to hear.

Avoid: Saying you have never breached. Saying you would work overtime to fix it alone. Claiming you would hide it.

Follow-ups: What is the difference between response and resolution SLA? Can an SLA be paused, and when is that legitimate? Have you ever had a breach, and what happened?

Q4. What makes a ticket L2 rather than L1?

Answer: L1 handles anything with a documented fix — access, password resets, known errors with a KB article, standard requests. It comes to L2 when there is no known fix, when it needs log analysis or a database check, when it involves application behaviour rather than user error, or when the impact is wide enough that it needs investigation rather than a workaround. On our account L1 also escalates anything that touches CMDB or change.

Tip: Give the boundary as a rule, then an example. "A user cannot log in" is L1. "Twenty users in one region cannot log in since last night's release" is L2.

Follow-ups: Have you ever pushed a ticket back to L1? Who decides the tier — the tool or a person?

Q5. What do you include when you escalate to L3?

Answer: What the issue is, when it started, how many users are affected, the exact error and log extract, what I have already checked and ruled out, whether it correlates with any recent change, and the business impact. The most important part is what I have ruled out, because that stops L3 repeating my work.

Tip: This answer is a strong signal of L2 maturity. Say the "ruled out" line explicitly.

Follow-ups: Give me a real example. What if L3 sends it back?

Q6. Give me an example of a ticket you escalated to L3. What had you already eliminated?

Answer: Use a real one. Structure: the symptom, the checks you did (logs, database, recent changes, whether it was one user or many), what each check eliminated, then what remained that needed L3's access or code knowledge.

Tip: Prepare two of these. This is the single most likely technical question in the whole interview.

Follow-ups: How long did it take L3 to fix? What was the root cause? Did you write a KB article after it?

Q7. Tell me about the CMDB work you do.

Answer: I keep configuration items and their relationships accurate in my scope — updating CIs when something is added, moved or decommissioned, correcting relationship records so upstream and downstream dependencies are right, and cleaning up duplicates. The reason it matters is change impact: if the relationships are wrong, a change looks low risk when it is not.

Tip: Always finish with *why* CMDB matters. Anyone can say they update records. Connecting it to change risk shows understanding.

Follow-ups: What CI classes? How do CIs get created? What is Discovery? What is an identification rule?

Q8. How do configuration items get into your CMDB?

Answer: Three ways generally: Discovery, which scans the network and populates automatically; bulk import through import sets and transform maps; and manual creation for things Discovery cannot see. On our account [state which applies to you]. Duplicates are the main risk, which is what identification and reconciliation rules exist to prevent.

Tip: If your account does not use Discovery, say so. Do not describe a process you have not seen.

Follow-ups: What is a reconciliation rule? What happens when two sources claim the same CI?

Q9. What is a CI relationship and why does it matter?

Answer: It records how CIs depend on each other — an application runs on a server, a server sits in a data centre, a service depends on a database. It matters because impact analysis uses it. If you take a database down for patching, the relationships tell you which applications and which business services go down with it. Wrong relationships mean a change gets approved as low risk and then causes an outage.

Follow-ups: Upstream versus downstream? How does this feed into a change record?

Q10. Walk me through your monitoring responsibilities.

Answer: AppDynamics for application performance — response times, error rates, and alerts on thresholds. Control-M for the batch schedule — job failures, jobs running long, and dependency chains where one failure blocks others. Informatica for data load failures. When something alerts I check whether it is a real failure or a transient, look at whether anything changed recently, and either fix it if it is within my scope or escalate with the evidence.

Tip: Distinguish "failed" from "stuck" for Control-M. That detail lands well.

Follow-ups: What was your last real alert? What is your escalation path at 2am? What is a false positive and how do you handle repeated ones?

Q11. Which of your five listed tools are you strongest and weakest in?

Answer: See **Danger 3** in Part 1. Name two you own, be honest about the rest.

Q12. You mention SQL on Oracle. What do you actually use it for?

Answer: Mostly investigation. When a user says a value on screen is wrong, I query the underlying table to see what the data actually is, which tells me whether it is a data problem or a display problem. I also use it to check whether a batch load inserted the expected number of rows, and to find duplicates. Read-only in production; anything that changes data goes through change approval.

Tip: That last sentence is important. It shows you understand production discipline.

Follow-ups: Write a query for me. Have you ever run an UPDATE in production? What is the approval process?

Q13. Tell me about the Excel reporting you do.

Answer: I build reports on ticket trends and SLA compliance that the team uses in daily and weekly reviews — ticket volume by category, ageing, SLA met versus breached, and repeat issues. I use macros and VBA to automate the repetitive formatting and refresh so it does not need rebuilding each week.

Follow-ups: Why Excel and not a ServiceNow dashboard? Did you write the macros yourself? What decision came out of one of those reports?

Q14. You wrote knowledge base articles. Tell me about one.

Answer: Be honest that it is occasional, then give one real example: the recurring issue, what the article contains (symptom, cause, step-by-step resolution, when to escalate), and the outcome — L1 closing that issue without escalating to you.

Tip: The value framing is what matters: a good KB article reduces *your* workload by pushing resolution down to L1.

Follow-ups: How many have you written? How do you keep them current? Who reviews them?

Q15. Your only listed achievement is from a 2-month internship in 2023. What have you achieved at Cognizant?

Answer: Do not be caught by this. Have one prepared — see section 1.6. A recurring incident permanently resolved, a CMDB clean-up, a report the team adopted, a KB article that cut escalations, or being trusted with L2 within months of joining.

Tip: "Being given L2 ownership within [X] months of joining, when the usual path is longer" is a legitimate achievement if true.

Q16. You completed Administration Fundamentals last month. What did it cover and what did you build?

Answer: The course covers the platform basics — tables and forms, UI policies and data policies, business rules and client scripts, ACLs, update sets, import sets, reports and dashboards, and the ITSM applications. Then say what you actually practised on your PDI.

Tip: If you have not built anything, say "the course was mostly guided exercises and I am now building a catalog item with approvals on my PDI to consolidate it." Do not invent a project.

Follow-ups: Why Fundamentals and not CSA directly? When is your CSA? What is the hardest concept in it?

Q17. Why have you not done ITIL certification yet?

Answer: Honest and forward-looking: "My account runs ITIL-aligned incident, problem and change process so I have worked inside the framework daily, but I prioritised the platform certification first because that is the tool I use every hour. ITIL Foundation is my next one after CSA in November."

Tip: Never imply you hold it. This is a verifiable claim.

Q18. You did full-stack development in 2023. Why the change to support?

Answer: "The internship was two months of web development and I enjoyed it, but the Cognizant training showed me a different thing — that production systems break in ways nobody designed for, and finding out why is a genuine skill. Development knowledge helps me in support. I can read code, read logs and write SQL, which means I get to a cause faster than someone who only knows the ticket tool."

Tip: Frame development as an asset for support, not as your real ambition. See **Danger 2**.

Q19. Your BCA CGPA is 9.52 but Class X was 74 percent. What changed?

Answer: "School was general subjects and I was average at some of them. In BCA I was studying computers, which is what I actually wanted to do, and the results followed. It is the clearest evidence I have that I perform well when I am interested in the subject."

Tip: Do not be embarrassed. An upward trend is a good story.

Q20. What is your notice period, and when can you join?

Answer: Give the exact number from your Cognizant HR, plus whether buyout is possible. Never guess.

PART 4 — ROLE-BASED QUESTIONS (Q21–Q32)

Q21. What does Service Desk Management mean to you? (*Full treatment — this is the JD's must-have skill*)

Simple answer:

"It is running the single point of contact between users and IT so that issues get logged, prioritised, resolved or escalated properly, and nothing gets lost. It covers incident, request, problem and change process, SLA tracking, escalation, and keeping knowledge documented."

Detailed answer:

"I think of it in three layers.

The first is process. Every contact from a user becomes a record with the right category and priority, so that incidents, service requests, problems and changes are handled by the right process rather than all treated the same way.

The second is measurement and accountability. SLAs, escalation paths, ownership. A service desk without SLA tracking is just a mailbox. Somebody has to be answerable for whether the target was met and why it was not.

The third, and the part people underrate, is knowledge. If the same issue arrives twenty times and there is no article, you are paying twenty times to solve the same problem. Good service desk management pushes resolution down to the lowest tier that can handle it, and that only happens through documentation.

Underneath all of it, the point is service continuity. The user does not care which tier fixes it. They care that their business process is working."

Tips: Three-layer structure is easy to remember and sounds considered. That last line about the user is worth saying — it shows business perspective, which is what the client round tests.

Avoid: Defining it as "raising and closing tickets." Reciting ITIL definitions word for word. Talking only about tools.

Follow-ups: How do incident and problem management differ? What is the point of a CMDB in this? Where does voice support fit?

Q22. What is the difference between incident, problem, request and change? (*Full treatment — near guaranteed*)

Simple answer:

"An incident is something broken now — restore service fast. A problem is the underlying cause of one or more incidents — fix it permanently. A service request is a standard ask like access or a new laptop, not a fault. A change is a planned modification to the environment, which needs approval and risk assessment."

Detailed answer:

"The distinction is purpose, not size.

Incident — an unplanned interruption or degradation. The goal is restoring service as quickly as possible, and a workaround counts as success. If a payment screen is down and I get users onto an alternate route, the incident can be closed even though the fault is still there.

Problem — the cause behind incidents. Different goal: permanent removal, not speed. This is where root cause analysis and known error records live. The payment screen failing every Monday morning is a problem record, even though each Monday is a separate incident.

Service request — a standard, pre-approved ask. Nothing is broken. Access, software installation, a report. These usually run through a catalog item with a defined fulfilment path and often an approval.

Change — a planned addition, modification or removal in the environment. Needs a change record, risk and impact assessment, approval through CAB for anything non-standard, and a rollback plan. This is where the CMDB earns its keep, because impact assessment depends on CI relationships being right.

In practice the discipline is not mixing them up. Fixing something in production because a user asked nicely, without a change record, is how outages happen."

Tips: The workaround point on incidents is what separates someone who has read a definition from someone who has worked the process. Give one concrete example per type.

Avoid: Saying a problem is just a big incident. Forgetting that a workaround can close an incident. Skipping change entirely.

Follow-ups: What is a known error? What is an emergency change? Who approves a standard change? What is the difference between a major incident and a P1?

Q23. How is priority decided?

Answer: Impact times urgency. Impact is how much of the business is affected — one user, a department, a whole site. Urgency is how quickly it needs fixing given the business context. A matrix combines them into P1 to P4. A single user with a broken laptop is low impact; a single user who is the only person who can release payroll on payday is low impact but high urgency, and the matrix has to allow for that.

Follow-ups: Who can change a priority? What is your P1 definition? Have you ever disagreed with a priority?

Q24. What is a P1 and what happens when one is raised?

Answer: A critical incident with major business impact, usually a full service outage or something affecting a large user base or revenue. Typically a bridge call is opened, technical teams join, someone owns communications to stakeholders, and updates go out on a fixed interval until service is restored. All other work stops for the people on it. Afterwards it goes to problem management for root cause.

Tip: If you have joined a P1 bridge, say so and describe your role, even if it was small.

Q25. Where does voice support fit into a service desk?

Answer: Voice is the fastest channel for both diagnosis and reassurance. On a ticket you get what the user chose to type; on a call you can ask the questions that actually narrow it down and see it happening in real time. It also matters for high-impact issues, where a worried user needs a person rather than a status update. In my role I speak to business users on calls to collect details and explain fixes, and I keep the ticket updated with what was discussed so the record is not just in someone's head.

Tip: That last point about documenting the call is what separates a professional from someone who just talks. Say it.

Q26. How do you explain a technical issue to a non-technical user?

Answer: Say what it means for them first, then the cause in plain terms, then what happens next and when. Not "the Informatica load failed with a constraint violation" but "yesterday's data did not load, so the report you are looking at is a day behind. The team is reloading it and it should be correct by 2pm. I will confirm when it is done." No jargon, no blame, and always a time.

Tip: This is the JD's "communicate with technical and non-technical stakeholders" line. Have this answer ready verbatim.

Q27. What is a knowledge base and why does it matter to you personally?

Answer: It is the documented resolution for known issues. It matters to me personally because every article that lets L1 close an issue is an escalation that does not reach my queue. Documentation is not admin overhead, it is workload reduction.

Q28. What is a shift handover and what goes into it?

Answer: Open tickets and their current state, anything close to SLA, active P1 or P2 situations with who is engaged, jobs that failed and whether they were rerun, and anything the next shift needs to watch. The test of a good handover is that the next person does not have to re-diagnose anything.

Q29. Are you comfortable with 24x7 rotational shifts?

Answer: Yes, and be specific: state what you currently work. "I already work rotational shifts on my account including weekends, so it is not new to me." If you have done night shifts, say so.

Tip: Answer without hesitation. Any wavering here on a 24x7 account is a problem.

Q30. How do you prioritise when three tickets need you at once?

Answer: Business impact first, then SLA clock, then effort. A P2 affecting a department beats a P3 affecting one person even if the P3 is older. If two are genuinely equal, I take the one closer to breach. And if all three are genuinely critical I tell my lead rather than silently letting one slip, because that is a resourcing decision, not mine alone.

Tip: The escalation-to-lead line is the mature part of this answer.

Q31. What would you do differently on a new account with a technology you do not know?

Answer: Read the existing KB and closed tickets first, because they show what actually breaks. Shadow the current L2 for a week. Learn the escalation matrix and who owns what. Be willing to work at L1 initially to learn the account. And be honest early about what I do not know rather than guessing on a production system.

Q32. What is the difference between application support and infrastructure support?

Answer: Application support is about the behaviour of the application — errors, data issues, functional defects, integration failures. Infrastructure is the layer underneath — servers, network, storage, OS. They overlap constantly, which is why the first question in any incident is which layer it lives in. If an application slows down, AppDynamics tells me whether it is the code, the database or the infrastructure, and that decides who owns it.

PART 5 — PROJECT-BASED QUESTIONS (Q33–Q44)

Your resume is operations-led rather than project-led, so panels will treat your *work* as the project. Frame each of these as a project with a start, actions and outcome.

Q33. Tell me about a challenging project or piece of work. *(Full treatment — listed as a must-prepare)*

Use **STAR**. Best candidate from your experience: a recurring incident you traced to root cause, a CMDB clean-up, or a batch failure with business impact.

Model answer (adapt with your real details):

Situation: "On our account a particular Control-M job was failing intermittently, roughly once a week, and each failure meant the next morning's report data was incomplete. L1 was rerunning it and closing the incident, so it kept coming back."

Task: "It was assigned to me as an L2 incident again, and rather than rerun it, I wanted to find out why it kept happening."

Action: "I pulled the failure timestamps for the previous two months and lined them up. The failures clustered on the same day of the week and around the same time. I checked the job's dependencies and found it was starting before an upstream data load had finished on days when that load ran long. I checked the logs on those dates to confirm the upstream job was still running. Then I documented what I had found and raised it as a problem record with the scheduling team, with the evidence attached."

Result: "The dependency was corrected so the job waits for the upstream completion instead of starting on a fixed time. The failures stopped. I also wrote a KB article so that if it recurs, L1 knows what to check before rerunning."

Tips: Pick something with a number in it. The strongest version of this story is one where you refused to accept the workaround and went for the cause, because that is exactly what the JD's "software detectives" line means.

Avoid: Choosing something where you were a bystander. Saying "we" throughout — the panel needs to know what *you* did. Ending without a result.

Follow-ups: How long did it take? Who else was involved? What would you do differently? How did you convince the other team it was their issue?

Q34. What was your role on your PDI, and what have you built?

Answer: Only answer this with something real. If you have built a catalog item with approvals, a Business Rule and a UI Policy, describe it end to end. If you have not, say the course exercises were guided and you are currently building your own to consolidate — and then actually do it before the interview.

Q35. Tell me about the Smart Summarize project. *(If you re-add projects to the resume)*

Answer: A Python and Streamlit application offering YouTube video summarisation, PDF summarisation, chat over a PDF, and slide generation, using the Google API. Be ready on: which parts you wrote, how you handled API failures and rate limits, how you chunked long inputs, and what you would improve.

Follow-ups: What happens if the API is down? How did you handle long documents? What was the hardest bug?

Q36. Tell me about the Real Estate Management System.

Answer: A web application for buying, selling and renting properties, with an instalment calculator that computes payment schedules from duration and interest percentage. Be ready on the database design, how you validated user input, and how you handled the calculation logic.

Q37. How does a college project differ from production work?

Answer: Strong answer, and worth preparing: "In a project I controlled everything and if it broke, I fixed it whenever. In production, other people's work depends on it, there is an SLA on how fast it gets fixed, you cannot change anything without approval, and you have to document what you did so the next person understands it. The technical part was the smaller adjustment. The discipline was the bigger one."

Q38. Describe a time you found the root cause of something nobody else had.

Answer: STAR. Emphasise the investigation method — correlating timestamps, comparing a working case with a failing one, checking what changed recently.

Q39. Have you worked on a change or release? What was your part?

Answer: Describe post-deployment validation if that is what you do: checking the application after a release, confirming key functions work, watching monitoring for new errors, and raising defects with evidence if something broke. If you have raised a change record yourself, say so and describe the risk assessment.

Q40. Tell me about a CMDB clean-up or data quality piece of work.

Answer: Situation (duplicate or stale CIs found), action (identified duplicates, traced which source created them, corrected relationships, retired decommissioned CIs), result (accurate impact analysis for change). Quantify if you can.

Q41. What reporting or automation have you built?

Answer: The Excel macro reporting. Explain what was manual before, what you automated, and time saved.

Q42. What project are you most proud of and why?

Answer: Pick work, not college. Choose the one where you changed an outcome rather than completed a task.

Q43. What would you do differently if you started that work again?

Answer: Always have a real answer. Good honest options: "I would have raised it as a problem record earlier instead of treating three incidents separately" or "I would have documented it while I was doing it rather than afterwards."

Tip: Never say "nothing." It reads as no self-reflection.

Q44. How do you handle work where you do not know the answer at all?

Answer: Check the KB and previously closed similar tickets first, because someone has usually seen it. Reproduce it if I can, because a reproducible issue is half solved. Read the logs and narrow down which component it is. Ask a colleague who owns that area, with what I have already tried. And escalate with evidence rather than guessing on production. The one thing I will not do is experiment on a live system to see what happens.

PART 6 — TECHNICAL QUESTIONS (Q45–Q92)

6.1 BEGINNER LEVEL (Q45–Q57)

These are screening questions. Getting one wrong here does real damage, because they are considered basic for your profile.

Q45. What is ITIL and what are the main ITSM processes you work with?

Answer: ITIL is a framework of best practices for IT service management — how to design, deliver, support and improve IT services. The processes I work with daily are incident management, problem management, change management and request fulfilment, plus configuration management through the CMDB. My account follows ITIL-aligned process. I am not ITIL certified yet; that is my next certification after CSA.

Q46. What is a CMDB?

Answer: A Configuration Management Database. It stores configuration items — servers, applications, databases, services, network devices — and the relationships between them. Its purpose is to give a single accurate picture of the environment so that impact analysis, change risk assessment and incident diagnosis are based on facts rather than guesswork.

Q47. What is a CI?

Answer: A Configuration Item — any component that needs to be managed to deliver a service. It has a class (server, application, database), attributes (name, owner, environment, status) and relationships to other CIs.

Q48. What is an SLA, an OLA and an underpinning contract?

Answer: An SLA is the agreement with the customer on service targets, such as resolving P2 incidents within four hours. An OLA is internal, between teams within the same organisation, such as L3 responding to an escalation within one hour. An underpinning contract is with an external vendor. The point of OLAs and UCs is that you cannot meet an SLA if the internal and vendor commitments underneath it do not add up.

Tip: Many juniors only know SLA. Knowing all three stands out.

Q49. In ServiceNow, what is the difference between an Incident and a Request?

Answer: An Incident record is for something broken — unplanned interruption. A Request, usually raised through the Service Catalog as a Request Item, is a standard pre-approved ask where nothing is faulty. They use different tables, different workflows and different metrics.

Q50. What is the difference between a UI Policy and a Client Script?

Answer: Both run on the client side in the browser. A UI Policy is configuration — you set conditions and it makes fields mandatory, read-only or visible, without code. A Client Script is JavaScript, used when you need logic a UI Policy cannot express, like calling a script include or doing a calculation. Best practice is UI Policy first, Client Script only when necessary, because configuration is easier to maintain and upgrade-safe.

Q51. What is the difference between a UI Policy and a Data Policy?

Answer: A UI Policy only applies in the browser form. A Data Policy applies at the data level, so it enforces rules regardless of how the record is created — form, import, web service or script. If you need a field to always be mandatory even on an import, that is a Data Policy, not a UI Policy.

Tip: This is a very common ServiceNow interview question and it separates people who did the course from people who just used the tool.

Q52. What is a Business Rule?

Answer: Server-side script that runs when a record is inserted, updated, deleted or queried. It can run before, after, async, or as a display rule. Used for things like setting field values, validating data, or triggering downstream actions.

Q53. What is an Update Set and what does it not capture?

Answer: An Update Set packages configuration changes so they can be moved between instances — dev to test to production. It captures configuration: business rules, UI policies, forms, workflows, tables and fields. It does **not** capture data records, scheduled job execution history, or anything in tables not marked for update tracking. Data usually has to move separately through import sets or a data load.

Tip: The "what it does not capture" part is where most candidates stop. Say it.

Q54. What is an ACL and what does it check?

Answer: An Access Control List controls who can read, write, create or delete at table or field level. It evaluates in order: the required role, then the condition, then the script. All applicable checks must pass. ACLs evaluate field-level before table-level for reads, and the most specific rule applies.

Q55. Basic SQL: what is the difference between INNER JOIN, LEFT JOIN and FULL OUTER JOIN?

Answer: INNER JOIN returns only rows with a match in both tables. LEFT JOIN returns all rows from the left table plus matches from the right, with NULLs where there is no match. FULL OUTER JOIN returns all rows from both, with NULLs on either side where there is no match. In support work LEFT JOIN is the useful one, because it shows you records that are *missing* their counterpart — which is usually the bug.

Q56. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping. HAVING filters after grouping, so it can use aggregate functions. You cannot use COUNT or SUM in a WHERE clause; that needs HAVING.

Q57. Name five Linux commands you use and what for.

Answer: `tail -f` to watch a log live while reproducing an issue. `grep` to search logs for an error string or a transaction ID. `df -h` to check disk space, which is a very common cause of unexplained failures. `ps -ef | grep` to check whether a process is running. `cd` and `ls -ltr` to find the newest log file in a directory. Also `less` for reading large logs without loading them into memory.

Tip: Tying each command to *why* you use it is much stronger than listing them.

6.2 INTERMEDIATE LEVEL (Q58–Q74)

Q58. Write a SQL query to find duplicate records in a table.

Answer:

```
SELECT email, COUNT(*)
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Then explain: to see the full duplicate rows rather than just the key, you join back to the table or use `ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_date)` and look at anything with row number greater than 1.

Q59. Write a query to find records in table A that have no matching record in table B.

Answer:

```
SELECT a.*
FROM orders a
LEFT JOIN invoices b ON a.order_id = b.order_id
WHERE b.order_id IS NULL;
```

Mention that `NOT EXISTS` is often better on large tables and that `NOT IN` is risky if the subquery can return NULLs, because it returns nothing.

Tip: That NULL warning on NOT IN is an experienced-person detail. It lands well.

Q60. How would you check whether last night's data load worked correctly using SQL?

Answer: Three checks. Row count for the load date against the expected volume or against the same day last week. A check for NULLs in columns that should never be NULL. And a check for duplicates on the business key. If all three pass, the load is probably good; if the count is right but there are duplicates, the load ran twice.

Q61. What is an index and when would it not help?

Answer: An index is a lookup structure that lets the database find rows without scanning the whole table. It does not help when the query returns a large proportion of the table anyway, when the column has very low cardinality, when the query applies a function to the indexed column, or when the leading column of a composite index is not in the WHERE clause. Indexes also slow down inserts and updates, so more is not better.

Q62. What is the difference between DELETE, TRUNCATE and DROP?

Answer: DELETE removes rows, is logged row by row, can have a WHERE clause and can be rolled back. TRUNCATE removes all rows quickly, is minimally logged, resets storage, and cannot be filtered. DROP removes the table structure entirely. In production support I would rarely use any of them without a change record and a backup.

Q63. What is a transaction, and what does commit and rollback mean?

Answer: A transaction is a unit of work that either completes fully or not at all. Commit makes changes permanent; rollback undoes them. This matters in support because a half-completed transaction is a common cause of inconsistent data, and because if I ever have to correct data I run the SELECT first, then the UPDATE inside a transaction so I can roll back if the row count is wrong.

Tip: "Run the SELECT first to check the row count" is exactly what a careful support engineer says. Include it.

Q64. In ServiceNow, explain the order of execution when a record is saved.

Answer: Client side first: UI Policies and onSubmit Client Scripts. Then the record goes to the server, where before Business Rules run, then the database operation, then after Business Rules, then async Business Rules. Data Policies and engine-level validation apply on the server regardless of the client path. Understanding this order matters when a field value is being overwritten and you need to know what is setting it.

Q65. What is an Import Set and a Transform Map?

Answer: An Import Set is a staging table where incoming data lands before it touches the real table. A Transform Map defines how staging columns map to target fields, including coalesce fields that determine whether a row is an insert or an update. This is how bulk CI loads into the CMDB usually work, and coalesce is the field that prevents duplicates.

Q66. What are identification and reconciliation rules in the CMDB?

Answer: Identification rules define how the system decides whether an incoming CI is the same as an existing one, using identifier attributes like serial number or FQDN. Reconciliation rules define which data source is authoritative for which attribute, so that if Discovery and an import disagree about an attribute, the trusted source wins. Together they are what stops the CMDB filling up with duplicates and flip-flopping data.

Tip: This is an advanced CMDB answer for your level. If you can give it, you look strong.

Q67. What is the difference between a Problem, a Known Error and a Workaround?

Answer: A Problem is the record for investigating an underlying cause. A Known Error is a problem where the cause is identified but not yet permanently fixed. A Workaround is the temporary method to restore service without fixing the cause. The known error record holds the workaround so that when the incident recurs, L1 can apply it immediately instead of escalating.

Q68. What is a change type breakdown — standard, normal, emergency?

Answer: Standard is pre-approved, low risk, repeatable, follows a documented procedure, no CAB needed. Normal goes through assessment, authorisation and usually CAB depending on risk. Emergency is for restoring service urgently, uses an expedited approval path such as an ECAB, and the documentation is completed retrospectively. The rule is that emergency should be rare; frequent emergency changes usually mean the planning process is broken.

Q69. In AppDynamics, what do you look at when an application is slow?

Answer: Start at the business transaction level to see which transaction is slow, because "the app is slow" is usually one or two transactions. Then look at the transaction snapshot to see where the time is spent — application code, an external call, or the database. If it is database time, get the slow query. Also check error rates and JVM or memory health, and compare against the same period last week so you know whether it is actually abnormal.

Tip: "Compare with last week" is a great line. It shows you distinguish a problem from normal behaviour.

Q70. A Control-M job failed. What is the difference between failed, stuck and long-running?

Answer: Failed means it ran and returned a non-zero exit code, so there is an error to read. Stuck usually means it is waiting on a condition or a resource — a predecessor job, a file that has not arrived, or a lock. Long-running means it is executing but past its normal window, which often points at data volume or a database issue rather than a code failure. They need different responses: a failure needs the log, a stuck job needs the dependency checked, a long-running job you often let finish while watching it.

Q71. A batch job failed at 2am. Walk me through what you do.

Answer: Check whether it is a real failure or a known transient. Read the job log for the actual error. Check whether anything upstream failed first, because in a dependency chain the reported failure is often a symptom. Check whether it is safe to rerun — a job that partially inserted data may need cleanup first, and reruns without checking are how you get duplicates. If it is safe and within my runbook, rerun and confirm. If not, escalate to the batch or application team per the on-call matrix, and notify the business contact if the morning reports will be affected.

Tip: The "is it safe to rerun" question is the mature part. Most juniors say "rerun it." Say why you check first.

Q72. Users report the application is slow but monitoring shows green. What do you do?

Answer: Believe the users and question the monitoring. Establish scope first — which users, which location, which function, since when. If it is one region, it is likely network. If it is one function, monitoring may not have a transaction defined for it, which is a monitoring gap worth raising. Check whether the slowness is at the front end, the network or the back end, and try to reproduce it myself. Green dashboards often mean the thresholds are set too loosely or the affected path is not instrumented.

Q73. How would you investigate a data mismatch reported by a business user?

Answer: Get a specific example — the record ID, the value they expect and the value they see. Query the source table to establish what the data actually is. That single step splits the problem: if the database matches what the user sees, it is a data or process problem upstream; if the database is correct and the

screen is wrong, it is an application or caching problem. Then check whether the record went through a recent load or change, and whether other records in the same batch are affected or just this one.

Tip: Explaining that the query *splits* the problem into two branches is exactly the "software detective" thinking the JD is asking for.

Q74. What is the difference between a memory leak and high memory usage?

Answer: High memory usage can be normal for a busy application. A leak is memory that is allocated and never released, so usage climbs steadily and does not come back down after load drops, eventually causing out-of-memory errors or a restart. The signature is the shape of the graph over days, not the absolute number. A restart temporarily fixes a leak, which is why "it works after a restart and fails again in a week" is a strong leak indicator.

6.3 ADVANCED LEVEL (Q75–Q84)

Q75. Your CMDB shows an application depending on a server that was decommissioned six months ago. What is the impact and how do you fix it?

Answer: The impact is that change impact analysis is wrong in both directions — changes to that server look like they affect a live application, and the application's real dependency is unrecorded, so a change to the actual server looks safe. Fix: confirm with the infrastructure team what the application actually runs on now, retire the stale CI rather than deleting it so history is preserved, create the correct relationship, and then look at why it was not updated. Usually the root cause is that the decommissioning process does not include a CMDB step, which is a problem record, not just a data fix.

Tip: Finishing on "why did the data go stale" rather than just "I corrected it" is what makes this an advanced answer.

Q76. Two data sources disagree about the same CI's attribute. How do you resolve it?

Answer: This is what reconciliation rules exist for. Determine which source is authoritative for that specific attribute — Discovery is usually authoritative for technical attributes it can scan, while a business system may be authoritative for owner or cost centre. Configure the reconciliation rule accordingly rather than manually correcting the record, because a manual fix will be overwritten on the next import. If neither source should own it, mark it as manually maintained.

Q77. Same incident recurring every month. How would you drive it to permanent closure?

Answer: Raise a problem record and stop treating each occurrence as unrelated. Gather evidence across occurrences: timestamps, what was happening around each, and what changed before the first one. Look for a pattern in time, volume or user population. Identify root cause, then propose a permanent fix through change management with the owning team. In the meantime document the workaround in a known error record so L1 can restore service without escalating. Track it until the fix is deployed, then verify by monitoring for recurrence for at least a cycle longer than the previous interval.

Q78. How would you reduce ticket volume on your account by 20 percent?

Answer: Start with data, not opinion. Pull three months of tickets and group by category and root cause. Volume almost always concentrates: a handful of issue types generate most of the tickets. Then attack them by type — recurring faults go to problem management for permanent fix, repetitive requests get automated or turned into catalog items with self-service, and issues that L1 escalates unnecessarily get KB articles so they are resolved at first contact. Also check the reopen rate, because reopened tickets are hidden volume caused by closing things prematurely.

Tip: This is a favourite managerial-round question. The key move is "measure first."

Q79. What is the difference between MTTR, MTBF and first contact resolution, and which would you improve first?

Answer: MTTR is mean time to restore — how long incidents take. MTBF is mean time between failures — how often things break. FCR is the proportion resolved at first contact without escalation. If the desk is drowning, FCR first, because it is the cheapest to move and it reduces load on every tier above L1. If the business is unhappy, MTTR, because that is what they feel. MTBF is the long game and it belongs to problem management, not the service desk alone.

Q80. A change was approved as low risk and caused an outage. What went wrong and what do you do?

Answer: Immediate priority is service restoration — invoke the rollback plan if there is one, raise the P1, communicate. Afterwards the question is why the risk assessment was wrong, and the usual answer is CMDB relationships were incomplete, so the impact analysis missed a dependency. That becomes a problem record covering both the technical cause and the process gap. The corrective action is usually improving CI relationship accuracy for that service and reviewing how risk is assessed for that change type.

Q81. How do you handle a false positive alert that fires every night?

Answer: Do not just acknowledge and move on, because that trains the team to ignore alerts, and an ignored alert eventually hides a real one. Determine why it fires — usually a threshold set for a different workload, or a maintenance window not excluded from monitoring. Then fix it properly: adjust the threshold based on actual baseline data, add a maintenance window suppression, or change the condition to require sustained breach rather than a single spike. Raise it as a problem if the monitoring team owns the config.

Tip: Alert fatigue is a real operational concept. Naming it shows maturity.

Q82. Explain how you would trace a transaction across multiple systems.

Answer: Find a correlation identifier — an order number, transaction ID, or a trace ID if the systems propagate one. Then follow it through each hop: the front-end log, the application log, the integration or middleware log, and the database. At each hop establish whether the transaction arrived and whether it left. The point of failure is the first hop where it arrived but did not leave, or never arrived at all. Without a correlation ID this becomes timestamp matching, which is far slower and why correlation IDs matter.

Q83. What would you check if an application works for some users but not others?

Answer: Anything that differs between the two groups. Role and permission is first, since ACL or group membership differences are the most common cause. Then location and network path, browser or client version, data — the failing users may have records the working users do not, such as a NULL in a field the code does not handle — and caching. The method is to find one working and one failing user and compare them attribute by attribute until you find the difference.

Q84. When would you refuse to apply a fix in production?

Answer: When there is no change record and the fix is not a documented standard change. When I cannot explain what the fix does and how to reverse it. When the request came informally rather than through process, even from someone senior. When there is no rollback plan for something with wide impact. The exception is a genuine major incident, where an emergency change path exists — but that is a defined process, not an absence of process.

Tip: This question is testing whether you have production discipline. A candidate who says they would do whatever the business asked is a risk.

6.4 TROUBLESHOOTING SCENARIOS (Q85–Q92)

For each of these, follow the same method out loud: **establish scope, gather evidence, form a hypothesis, test it, escalate with what you ruled out.**

Q85. Users cannot log in to the application. Where do you start?

Scope first: all users or some, one location or all, since when, and did anything change last night. All users plus a recent release points at the release. Some users points at permissions or group membership. One location points at network. Then check whether the authentication service and the application are both up, look at the application and auth logs for the actual error, and confirm whether it is authentication failing or the session being rejected afterwards.

Q86. The application is up but a specific report shows no data.

Check whether the report ran, then whether the underlying data exists. Query the source table for the report's date range. If there is no data, the load failed or ran late — check Control-M and Informatica for that job. If the data exists but the report is empty, it is the report's filter, parameters or permissions. This is the classic case where a SQL query saves an hour of guessing.

Q87. Disk is full on an application server at 3am.

Confirm with `df -h` which mount. Find the biggest consumers, usually logs or temp files. Do not delete blindly — an active log file being removed can leave the process writing to a deleted handle and still not release the space. Archive or truncate per the runbook, restart only if the runbook says to, and then raise a problem record, because a disk filling up is a symptom of missing log rotation, not a one-off.

Q88. A scheduled job ran twice and created duplicate records.

Stop it from running again. Establish the scope of duplication with a GROUP BY on the business key. Determine whether the duplicates are harmless or whether downstream systems already consumed them. Cleanup goes through change approval with a tested SELECT before any DELETE, and a backup first. Root cause is usually a manual rerun without a check, or a scheduler misfire, and the permanent fix is idempotency — the job should detect it has already run for that period.

Tip: The word "idempotent" is worth knowing here.

Q89. An interface between two systems has stopped, and nobody noticed for two days.

The technical fix is one thing; the more important finding is that there was no monitoring on it. Establish what is missing, whether it can be replayed or has to be re-sent, and the business impact of two days of missing data. Then the problem record covers the monitoring gap, because the same silence will happen again otherwise.

Q90. A P1 is raised and you are the only person on shift.

Follow the major incident process rather than trying to fix it alone. Confirm and classify the impact, raise the P1 and open the bridge, engage the on-call teams per the escalation matrix, and take ownership of communication until someone senior joins. My job in the first ten minutes is mobilising the right people and giving the business an accurate impact statement, not personally debugging.

Tip: This answer is testing whether you know your limits. Trying to be a hero here is the wrong answer.

Q91. The user says it is urgent, but by the matrix it is a P3.

Ask what the business consequence is, because the matrix may genuinely be missing context — a single-user issue can be high urgency if that user is blocking a payroll run or a client deadline. If the context justifies it, raise the priority through the proper route with the justification recorded. If it does not, explain the ranking honestly, tell them where it sits and when it will be picked up, and offer a workaround if there is one. What I will not do is quietly reprioritise to keep someone happy, because that pushes someone else's genuine P2 back.

Q92. You applied a fix, closed the ticket, and it reopened the next day.

Own it without defensiveness. It means I treated a symptom, not a cause, or I closed without proper verification. Reopen, investigate properly this time, and check whether it should have been a problem record from the start. Then look at my own closure practice — was there a verification step I skipped, and did I confirm with the user rather than assuming.

PART 7 — BEHAVIOURAL QUESTIONS (Q93–Q108)

All of these use **STAR**: Situation, Task, Action, Result. Keep Situation short, spend most of your time on Action, and never skip Result.

Q93. Tell me about a mistake you made and what you learned. (*Full treatment — must prepare*)

Simple answer:

"Early on I closed a ticket after applying a workaround, without confirming with the user that it was actually working for them. It reopened the next day. Since then I always confirm with the user before closing, and if I have only worked around something, I say so in the ticket instead of writing 'resolved'."

Detailed answer:

Situation: "In my first few months I was picking up incidents and my focus was on closing them within SLA."

Task: "I got an incident where a user could not generate a particular report. I found a workaround, ran it, saw the report generate, and closed the ticket as resolved."

Action: "The next morning it was reopened. The workaround had produced the report for that one date, but the underlying issue was still there and the user needed it daily. I had also written 'resolved' rather than describing the workaround, so when it came back, the next person had no record of what I had actually done. I reopened it, documented properly what I had tried, and escalated it to L3 with the details."

Result: "Two things changed permanently in how I work. I confirm with the user that it is working for them before closing, not just that it worked once for me. And I distinguish in the ticket between a fix and a workaround, because writing 'resolved' when you mean 'worked around' misleads the next person and skews the reporting. My reopen rate went down after that and my lead noticed the notes got better."

Tips: Pick a real, modest mistake with a clear lesson and a behaviour change. Process mistakes are safer than mistakes that caused an outage. Always finish on what you do differently now.

Avoid: "I have not made any mistakes." A fake-humble non-mistake like "I work too hard." A mistake that suggests carelessness with production data or dishonesty. Blaming anyone else.

Follow-ups: Has it happened again? How do you verify closure now? What is your reopen rate?

Q94. Describe a situation where you handled conflict. (*Full treatment — must prepare*)

Simple answer:

"I had a disagreement with another team about whether an issue belonged to them or to us. Instead of arguing over ownership, I collected the log evidence showing where the failure actually happened, shared it with them directly, and we agreed in ten minutes. The lesson was that evidence settles ownership arguments faster than escalation does."

Detailed answer:

Situation: "A recurring batch failure was being bounced between our team and the scheduling team. Each of us believed it was the other's issue, and it had been going back and forth across shifts for several days."

Task: "It landed with me and I wanted it to stop being a ticket that just moved around."

Action: "Rather than reassign it again with a comment, I gathered evidence. I pulled the failure timestamps, the job logs and the upstream job's run times, and lined them up to show that our job was starting before their upstream load had completed on the days it failed. Then instead of putting that in the ticket and reassigning, I messaged their team member directly and walked him through it. I was careful to present it as 'here is what the logs show' rather than 'this is your problem'."

Result: "He agreed within about ten minutes because the evidence was clear rather than a claim. The dependency was corrected and the failures stopped. What I took from it is that most inter-team conflict in support is not really disagreement, it is two teams with incomplete information. Whoever assembles the full picture ends the argument."

Tips: Choose a professional disagreement over ownership or approach, not a personality clash. Show that you went to the person rather than escalating first. Land on evidence over opinion.

Avoid: Making the other person look bad. Saying you escalated to your manager immediately. Choosing a conflict you won by being forceful. Saying you have never had a disagreement — that reads as either inexperience or avoidance.

Follow-ups: What if he had disagreed with your evidence? When would you escalate to your manager? Have you had a disagreement with your own lead?

Q95. Describe a time you handled a tight deadline. (*Full treatment — must prepare*)

Simple answer:

"During a month-end close, a data load failed and the finance team needed their reports by 10am. I found the failure cause, got the load rerun after checking it was safe, and validated the row counts before telling them it was ready. It was done by 9:30. The key was telling them early where things stood rather than going quiet while I worked."

Detailed answer:

Situation: "Month-end reporting day. An overnight Informatica load had failed, which meant the finance team's reports would be a day behind, and they had a fixed 10am deadline with their own stakeholders.

Task: "I picked it up at the start of shift, around 7am. Three hours, and I did not yet know the cause.

Action: "First thing I did was tell the finance contact where things stood and that I would update them at 8:30 whether or not it was fixed, so they were not chasing me and could plan. Then I worked the problem: read the load log, found the failure, checked whether the upstream file had arrived complete, and confirmed whether a partial load had inserted rows that would duplicate on rerun. It had not, so a rerun was safe. I got it rerun, then validated the row count against the previous month before declaring success, because handing them wrong data on time would have been worse than late data.

Result: "Reports were available at 9:30. What I would highlight is the communication, not the speed. Giving them a fixed update time meant they were not calling me every twenty minutes, which is what actually made the three hours workable."

Tips: The strongest deadline stories are about communication and prioritisation, not heroics. The "validated before declaring success" point matters — it shows you would not trade correctness for speed.

Avoid: Making it sound like you worked alone through the night. Saying you skipped verification to hit the time. Having no result.

Follow-ups: What if you had not fixed it by 10? Who else did you involve? How do you decide when to escalate versus keep working?

Q96. Explain stakeholder management in your work. (Full treatment — must prepare)

Simple answer:

"I deal with three kinds of stakeholder: business users who raised the ticket, other technical teams like L3 and infrastructure, and my own lead and the client contact. Each needs different information. Users need impact and timing in plain language. Technical teams need evidence. Leads need to know risk early. The main rule I follow is that nobody should learn bad news from a report."

Detailed answer:

"In practice it is three groups.

Business users. They need to know what it means for their work, when it will be resolved, and to be told when it changes. No jargon, and always a time even if the time is 'I will update you at 2pm'. Silence is what makes users angry, more than the fault itself.

Technical teams. L3, infrastructure, the batch team, vendors. They need precision and evidence: the error, the logs, the timeline, and what I have already eliminated. Sending an escalation without that is what causes tickets to bounce back and forth.

My lead and the client contact. They need early warning on anything that will show up as an SLA breach or a business escalation. My rule is that my lead should never find out about a problem from a monthly report. If I can see something going wrong, I flag it while it can still be influenced.

Underneath all three, the same principle applies: set an expectation and then meet it. An honest 'this will take two days' builds more trust than an optimistic 'by end of day' that slips."

Tips: The three-group structure makes this answer easy to deliver. The line about nobody learning bad news from a report is memorable and true.

Avoid: Treating all stakeholders the same. Only talking about users. Saying you avoid giving bad news.

Follow-ups: How do you handle a stakeholder who wants constant updates? What if the business contact escalates over your head? Give me an example of managing a difficult stakeholder.

Q97. Describe a leadership example.

Answer (STAR): With 13 months, leadership means initiative and influence, not managing people. Good options: you noticed a recurring issue and drove it to permanent fix across two teams; you wrote KB articles that changed how L1 handled something; you improved the shift handover format; you helped onboard a new joiner. Be explicit: "I did not have authority, so I had to do it by getting the evidence together and bringing people along."

Tip: Do not invent team management. "Leading without authority" is the honest and impressive version at your level.

Q98. Explain your biggest achievement.

Answer: Pick from work, not college. Best candidates: driving a recurring incident to permanent closure, a CMDB accuracy improvement, or being trusted with L2 ownership faster than the normal path. Quantify. Explain why it mattered to the business, not just that you did it.

Q99. Tell me about a time you worked in a team.

Answer: Use shift-based work — handovers, covering for colleagues, joint work on a P1. Emphasise that in a 24x7 team, quality of handover is teamwork, because the next person's success depends entirely on your notes.

Q100. Tell me about a time you helped a colleague.

Answer: A specific instance — walking a new joiner through the escalation matrix, sharing a query you had written, taking a ticket when someone was overloaded. Show the outcome for the team, not just your generosity.

Q101. Tell me about a time you disagreed with your manager.

Answer: Choose something professional and low-stakes, show you raised it privately with reasoning, and show you accepted the decision if it went the other way. "I put my case with the data. He had context I did not, and once he explained it I agreed. What I took from it was to ask about the wider picture before assuming my view was complete."

Tip: Never say you have never disagreed, and never say you went around them.

Q102. Tell me about a time you received difficult feedback.

Answer: Something concrete — ticket notes not detailed enough, closing tickets too fast, needing to escalate earlier. Show what specifically changed in your behaviour afterwards. The credibility is in the specificity of the change.

Q103. Tell me about a time you went beyond your role.

Answer: Taking on the monitoring rotation, writing KB articles that were not required of you, investigating a problem instead of applying the standard workaround. Frame it around outcome for the team.

Q104. Tell me about a time you had to learn something quickly.

Answer: Strong option: joining the account and having to learn a stack — ServiceNow, Control-M, AppDynamics, Informatica — in a short window. Describe the *method*: reading closed tickets to see what actually breaks, shadowing, and asking specific questions rather than general ones.

Q105. How do you handle pressure and long hours?

Answer: Concretely. Prioritise by business impact rather than trying to do everything, communicate early when something is going to slip, and escalate rather than sitting silently on a problem. Mention that rotational shifts have taught you to protect your sleep and handover properly, because a tired handover creates the next shift's problem.

Q106. Tell me about a time you had to say no.

Answer: A request to change something in production without a change record, or to reprioritise a ticket without justification. Show that you said no to the *action* while still helping with the *need* — offering the correct route rather than just refusing.

Q107. How do you handle repetitive work?

Answer: Honest and constructive: repetitive work is a signal. If I am doing something ten times, either it should be documented so L1 can do it, or it should be automated, or the underlying fault should go to problem management. That is how I got into KB writing and the Excel macro reporting. But some repetition is the job and I do it properly.

Tip: This is a hidden test for support roles. Saying you get bored is a red flag. Saying you look for the pattern is the right answer.

Q108. What motivates you at work?

Answer: Be specific and credible. "Solving something nobody else has managed to solve. The satisfaction in support is not closing tickets, it is the one where you find out why it kept happening and it stops. The Control-M dependency issue is the example — three people had been rerunning it for weeks, and finding the actual cause meant nobody had to deal with it again."

PART 8 — HR ROUND (Q109–Q131)

Accenture's HR round tests stability, fit, availability and expectations. It is not a formality; people get rejected here.

Q109. Walk me through your profile briefly.

Short version of 2.1. Sixty seconds, not ninety, because HR wants facts: current role, company, experience, location, education, certifications, notice period.

Q110. Why are you leaving Cognizant after only 13 months?

See **Danger 1**. This is HR's main question. Do not improvise it.

Q111. What are your strengths? (*Full treatment*)

Simple answer:

"Three things. I am methodical when I troubleshoot, so I work by elimination rather than guessing. I document properly, which sounds small but it is why my escalations do not come back. And I am comfortable saying I do not know something rather than pretending, which on a production system matters."

Detailed answer:

"I would name three, and I will give you evidence for each rather than just the adjective.

Structured troubleshooting. When something breaks I establish scope first — how many users, since when, what changed — before I start checking things. It sounds obvious but it is the difference between a two-hour investigation and a two-day one. The Control-M dependency issue I found was purely because I lined up timestamps instead of rerunning the job again.

Documentation discipline. My escalations to L3 include what I have already ruled out, and I write KB articles for recurring issues. The measurable effect is that things do not bounce back to me, and L1 stops escalating issues that now have an article.

Knowing my limits. I would rather say 'I have not done that, here is what I have done that is close' than bluff. In a support role, someone who guesses on a production system is dangerous. That applies to my tool depth too — I am strong on AppDynamics and Control-M, and I am upfront that Informatica and Azure are shallower for me."

Tips: Never give a strength without evidence. Three is the right number. That third strength is unusual and it lands well, because it also pre-answers the tool-depth cross-question.

Avoid: Generic lists — hardworking, quick learner, team player — with no example. More than three. Strengths irrelevant to the role.

Follow-ups: Which of those is most useful for this role? Give me another example of the second one. What do your colleagues say you are good at?

Q112. What are your weaknesses? (*Full treatment*)

Simple answer:

"My tool depth is uneven. I put five platforms on my resume and I am genuinely strong on two of them. I have been fixing that by going deeper rather than wider, which is why I did the ServiceNow certification instead of picking up another tool. The second one is that I used to over-investigate before escalating, trying to solve everything myself. I have learned to escalate earlier with good notes."

Detailed answer:

"Two real ones.

First, breadth without matching depth. I have exposure to AppDynamics, Control-M, Informatica, Salesforce and Azure, and my depth across those is uneven. I can work AppDynamics and Control-M properly; on the others my involvement is narrower than the resume line might suggest. I decided the answer was to go deep rather than add a sixth tool, which is why I put the time into the ServiceNow certification instead. It is still a gap and I would rather name it than have you find it.

Second, I used to hold on to tickets too long. Early on I saw escalating as admitting I could not do it, so I would spend an extra hour trying to crack something myself. That is bad for the SLA and bad for the user. My lead pointed it out. What changed is that I now judge it on whether I have a hypothesis I can still test — if I do not, more time will not help, and the right move is escalating with everything I have ruled out so L3 does not start from zero."

Tips: Give real weaknesses with a correction in progress. Two is enough. Both of these are safe because neither suggests you are careless or unreliable, and the first one disarms Danger 3 before it is asked.

Avoid: "I am a perfectionist" or "I work too hard" — every interviewer has heard these and they read as evasive. A weakness that is core to the job, like "I am not good with people" for a service desk role. Any weakness with no corrective action.

Follow-ups: How do you decide when to escalate now? What are you doing about the tool gap? What feedback have you had recently?

Q113. What are your salary expectations?

Answer: Do research first — check current Bengaluru ranges for application support with one year and ServiceNow skills. Then: "My current CTC is [X]. Given the role, the ServiceNow certification and the relocation to Bengaluru, I am looking at [range]. I am open to discussing the overall package rather than only the fixed component." Give a range, not a single number, and know your walk-away figure before the call.

Tip: Factor Bengaluru's cost of living against Pune. That is a legitimate part of your case.

Q114. What is your current CTC?

Answer: State it accurately. It will be verified through payslips. Never inflate — an inflated figure discovered at document verification can void the offer.

Q115. What is your notice period?

Answer: Exact number from your HR, plus whether buyout is possible and whether you would fund it. Do not guess.

Q116. When can you join?

Answer: Notice period plus a realistic buffer. Do not promise a date you cannot meet; a missed joining date damages you with the account.

Q117. Are you willing to relocate to Bengaluru?

Answer: "Yes. I trained and interned there on-site with Cognizant, so I know the city and the commute, and I have already relocated once from Shirpur to Pune for work." That last clause is your strongest evidence — you have relocated before, so this is not a hypothetical.

Q118. Are you comfortable with rotational and night shifts?

Answer: Yes, with specifics on what you currently do. Be unhesitating.

Q119. Are you interviewing elsewhere?

Answer: Honest but controlled: "I have a couple of processes in progress, but this is the role I am most interested in because of the troubleshooting scope." Do not name companies. Do not claim to have offers you do not have.

Q120. Do you have any offers in hand?

Answer: If yes, say so factually and without using it as leverage unless you actually would take it. If no, say no. A bluffed offer that gets called can end the process.

Q121. How did you hear about this role?

Answer: Straightforward. If through the careers site, say so and mention what in the description made you apply.

Q122. What do you know about Accenture?

Answer: Have three facts ready, checked in the week of your interview: scale and global delivery model, their managed services and application support business, and something recent about their ServiceNow or platform practice. Then connect one to your own work.

Q123. Why should we not hire someone with more experience?

Answer: "You might, for a senior role. For this one the description says 0 to 2 years, which tells me you want someone who can already work independently at the ticket level and still be shaped by the account. I am already L2, so I am not starting from zero, and I have not been doing it long enough to be rigid about how things must be done."

Q124. How do you handle criticism?

Answer: Concretely, with the ticket-notes or escalation-timing example from Q112 or Q102. Show a specific behaviour change.

Q125. What are your hobbies and interests outside work?

Answer: Answer genuinely and briefly. It is a rapport question, not a trap. If you do technical learning outside work — your PDI, certifications — mention it but do not make your whole life work, because that reads as either dull or as burnout risk on a 24x7 account.

Q126. Tell me about your family background.

Answer: Common in Indian HR rounds. Brief and factual — where you are from, what your parents do, siblings. Two sentences. It usually connects to relocation stability, so if you have family support for moving to Bengaluru, mention it.

Q127. How do you manage your MCA alongside full-time work and shifts?

Answer: See **Danger 4**. State the mode of study plainly and give evidence you have already sustained it for over a year.

Q128. What if we place you in a different technology or on a different account?

Answer: Show flexibility without being a pushover: "I am open to it. My preference is staying in the application support and ITSM space because that is where my certifications and experience compound. If the account needed a different tool stack, I would learn it — that is what I did with Control-M and AppDynamics, neither of which I knew a year ago. What I would want is to keep doing support and troubleshooting work rather than moving into an unrelated function."

Q129. Where else have you applied and why?

Answer: General, not specific. "I have been selective and looked at application support and ServiceNow-focused roles rather than applying broadly."

Q130. What would you do if you do not get this role?

Answer: "Keep building in the same direction. Finish CSA in November, add ITIL, and get deeper on the platform. I would also ask for feedback if you were willing to give it, because that is more useful to me than the outcome."

Q131. Do you have any questions for us?

Answer: Always have three ready. Good ones for this role:

- What does the support tier structure look like on this account, and where would I sit in it?
- What are the main technologies in the application stack I would be supporting?
- What does the shift pattern look like, and how large is the team?
- What does growth look like for someone in this role over two to three years?

Avoid asking: salary, leave policy or work-from-home in a first round. Anything answered on their careers page. And never say "no questions" — it reads as no interest.

PART 9 — MANAGERIAL ROUND (Q132–Q152)

The managerial round tests judgement, ownership and whether you can be trusted with a client's system without supervision. Answers should show you think about consequences beyond your own ticket.

Q132. You have three P2s and one P1 at the same time, alone on shift. What do you do?

Answer: P1 first, but not by working it alone. Raise it, open the bridge, engage on-call per the matrix, and own communications. Then tell my lead immediately that three P2s are unattended so it is a resourcing decision rather than me silently letting them breach. Put a holding update on each P2 so the users are not in the dark. The judgement point is that in an overload I escalate the *situation*, not just the tickets.

Q133. How do you decide what to escalate to your manager?

Answer: Anything that will become their problem later if they do not know now. Specifically: SLA breaches that are going to happen, a user or client escalation brewing, a repeated failure suggesting something structural, a request to bypass process, or my own capacity being exceeded. I do not escalate things I can decide myself — that wastes their time and mine.

Q134. How would you improve the service desk on your account?

Answer: Measure first. Pull three months of tickets, group by category and root cause, and look for concentration. Then three moves: recurring faults into problem management for permanent fix, high-volume simple requests into self-service or catalog automation, and issues L1 escalates unnecessarily into KB articles so they resolve at first contact. I would also look at the reopen rate, because reopened tickets are hidden volume that the raw count hides.

Q135. A team member is consistently writing poor ticket notes. What do you do?

Answer: Talk to them directly first, privately, and frame it around consequence rather than criticism — that when notes are thin, the next shift re-diagnoses from scratch and L3 sends things back. Offer a concrete standard or a template rather than just telling them to do better. If it does not change and it is affecting the team's SLA, then it goes to the lead, but only after I have actually tried.

Q136. How do you handle a colleague who is not pulling their weight?

Answer: Find out why before judging. It might be workload, a gap in knowledge on the account, or something personal. If it is knowledge, help. If it is workload distribution, that is a lead conversation. What I would not do is complain about it to the rest of the team, because that damages the shift dynamic more than the original problem.

Q137. What would you do in your first 30, 60 and 90 days here?

Answer: Structured and specific:

- **First 30:** learn the account. Read the KB and the last three months of closed tickets, because they show what actually breaks. Learn the escalation matrix, the tool landscape and who owns what. Shadow and take simple tickets.
- **30 to 60:** work the queue independently at my tier, start contributing KB articles for gaps I have hit, and get onto the monitoring rotation.
- **60 to 90:** own something. Pick a recurring issue and drive it to a problem record. Be the person others can hand a hard ticket to.

Tip: This is a favourite managerial question and most candidates answer it vaguely. A specific 30/60/90 makes you sound like someone who has thought about the job rather than the offer.

Q138. How do you handle a situation where you disagree with a process?

Answer: Follow it while raising it. Support processes usually exist because something went wrong once, so I would want to know the history before pushing. Then make the case with evidence — this is costing this much time, or causing this many reopens — through my lead rather than working around it. Working around a process quietly is how audit findings happen.

Q139. How do you prioritise between an SLA metric and doing the job properly?

Answer: Correctness wins, but they conflict less than people assume. Closing a ticket prematurely to hit an SLA produces a reopen, which hurts the metric anyway and costs the user more. Where they genuinely conflict — the fix needs more time than the SLA allows — the answer is escalating and being honest about the breach, not gaming the record. A breach with a documented reason is defensible. A closed ticket that was not fixed is not.

Tip: This is a values test. Never suggest you would manipulate the metric.

Q140. What would you do if you were asked to close tickets you had not actually resolved?

Answer: I would not, and I would say why: it misrepresents service quality to the client, it means the user's issue is still live with no record, and it is the kind of thing that surfaces in an audit or a client review. If it came from a lead under pressure, I would raise the underlying pressure with them — if we cannot meet the volume, that is a capacity conversation, not a data conversation. If it persisted I would escalate it.

Tip: This tests integrity. Answer without hedging.

Q141. How do you handle being on call?

Answer: Practically: know the runbook and escalation matrix before the shift so I am not reading it at 2am, keep the phone reachable, respond within the agreed window, and know the difference between what I can act on and what I must wake someone else for. Also handover properly afterwards so the day shift knows what happened overnight.

Q142. How do you deal with monotonous work in a support role?

Answer: See Q107. Repetition is a signal — automate it, document it so it moves down a tier, or fix the cause. Some of it is simply the job and I do it properly regardless.

Q143. How do you keep learning while working shifts?

Answer: Concretely: I did Administration Fundamentals last month around shift work, my CSA is booked for November, and my MCA runs alongside. The method is small consistent blocks rather than waiting for free weekends, which never arrive on a rotational roster.

Q144. What kind of manager do you work best with?

Answer: Diplomatic and honest: "Someone who tells me clearly what good looks like and then lets me get on with it, and who is available when I escalate. I would rather have direct feedback than polite feedback." Avoid describing your current manager negatively.

Q145. How do you handle unclear instructions?

Answer: Ask before starting, and ask specifically rather than "can you explain" — restate what I think is being asked and confirm. On production work, guessing is expensive. If the person is not available and it is not urgent, I would document my assumption and confirm before acting.

Q146. Tell me about a time you took ownership of something outside your remit.

Answer: STAR. The Control-M dependency story works well here, because you drove it across a team boundary without authority.

Q147. How would you handle an SLA breach that was your fault?

Answer: Report it before it is discovered, with the reason and what I would do differently. Then fix the cause in my own practice. Trying to hide it is worse than the breach, because it costs credibility that takes far longer to rebuild.

Q148. How do you deal with conflicting instructions from two seniors?

Answer: Do not pick one silently. Go back to both — or to the more senior — state the conflict plainly and ask for a decision. Then document the outcome. Choosing one quietly means someone will be surprised later, and it will be my problem.

Q149. How do you measure whether you are doing a good job?

Answer: Not just ticket count. SLA compliance, reopen rate, how often my escalations come back from L3, whether the KB articles I write get used, and whether colleagues bring me the hard tickets. That last one is the real signal.

Q150. What is more important — process or outcome?

Answer: Outcome is the point, process is how you get it reliably and repeatably. In production support, process is what stops one person's clever shortcut becoming next month's outage. Where the process genuinely obstructs the outcome, the answer is changing the process, not ignoring it.

Q151. Would you take on a team lead role if offered?

Answer: Yes, with honesty about readiness: "Not immediately. I want another year or two of technical depth first, because I do not want to lead a support team without being able to solve what they escalate. After that, yes, and I have been building towards it by taking ownership of problems and mentoring where I can."

Q152. How do you handle a client who blames the team unfairly?

Answer: Do not get defensive and do not throw the team under the bus. Establish facts calmly, share what the record actually shows, and acknowledge anything that genuinely was our miss. Then move it to what happens next. Defending the team is done with evidence, not with argument.

PART 10 — CLIENT ROUND (Q153–Q173)

The client round tests business awareness, communication and whether they would be comfortable with you talking to their users. Drop the jargon here.

Q153. Tell me about a difficult client or user interaction. (*Full treatment — must prepare*)

Simple answer:

"A user was frustrated because an issue affecting their daily report had been open for a few days and kept being passed around. I called them instead of updating the ticket, let them explain, told them honestly what had gone wrong on our side, gave them a specific update time, and stuck to it. The issue took another day to fix, but they were fine with it because they knew what was happening."

Detailed answer:

Situation: "An incident affecting a daily report had been reassigned between teams a few times over several days. The user had raised it twice and was clearly frustrated by the time it reached me."

Task: "I needed to fix the issue, but first I needed to fix the relationship, because they had stopped trusting the updates."

Action: "I called rather than replying on the ticket. I let them explain the impact fully without interrupting or defending. Then I was honest: the ticket had been bounced between teams and that was our failure, not their fault for raising it again. I did not blame the other team by name. I told them exactly where the investigation stood, what I was going to check, and that I would call them at a specific time whether or not I had an answer. Then I actually called at that time, even though I only had partial progress."

Result: "The fix took another day. But the tone changed completely after that first call, and they told my lead afterwards that the communication had been the difference. What I learned is that users escalate mostly because they feel ignored, not because of the elapsed time itself."

Tips: Choose frustration over abuse. Show that you called rather than emailed. Admit the process failure honestly. The closing insight is the most valuable part — say it.

Avoid: Making the user sound unreasonable. Blaming a named team or colleague. Claiming you fixed it immediately, which is not believable. Saying you escalated to your manager to handle the user.

Follow-ups: What if they had still been angry after the call? What if the fault genuinely was on their side? How do you say no to a client?

Q154. How do you explain a delay to a business user?

Answer: Impact, honest reason, next step, and a time. "The data load failed overnight so today's figures are a day behind. The team is reloading and I expect it correct by 2pm. I will confirm when it is done." No jargon, no excuses, and never a vague "we are working on it."

Q155. A client asks for something outside the agreed scope. What do you do?

Answer: Do not refuse flatly and do not agree on the spot. Understand what they actually need, because sometimes the need can be met within scope in a different way. Then be honest that it falls outside the current scope and route it properly through my lead or the account manager, who own the commercial side. What I would not do is quietly absorb extra work, because that sets an expectation nobody agreed to and hides a scope problem.

Q156. How would you handle a client who wants a P3 treated as a P1?

Answer: Ask what the business consequence is, because they may have context the matrix does not capture. If it justifies a higher priority, raise it through the proper route with the reason recorded. If it does not, explain the ranking honestly — including that raising this one pushes back someone else's genuine higher-priority issue — tell them where it sits and when it will be picked up, and offer a workaround if one exists.

Q157. What would you do if you made an error that affected the client?

Answer: Report it immediately, before it is found. State what happened, the impact, what has been done to contain it, and what will prevent it. Do not minimise and do not over-apologise past the point of usefulness. Clients generally forgive errors that are disclosed early and handled properly; what they do not forgive is finding out later.

Q158. How do you build trust with a client team?

Answer: Small commitments, reliably met. Say you will update at 3pm, update at 3pm. Be honest about what you do not know. Never promise a timeline you are not confident of, because one missed promise costs more than several honest "I do not know yet's".

Q159. How would you present your monthly SLA numbers if they were bad?

Answer: Lead with the number, not the excuse. Then the breakdown — which tickets breached and why, separating genuine capacity or complexity issues from process failures. Then what changes next month. Clients respect an honest bad number with a plan far more than a good number that turns out to be massaged.

Q160. A client user is rude to you on a call. How do you respond?

Answer: Stay professional and do not match the tone. Acknowledge the frustration, keep bringing it back to the facts and the next step. If it becomes abusive rather than just angry, stay calm, end the call politely and escalate to my lead — I do not have to absorb abuse, but I also do not escalate someone simply for being annoyed.

Q161. How do you say no to a client?

Answer: Explain the reason, not just the refusal, and always offer the alternative route. "I cannot change that in production without a change record, because it would go in untested and there would be no rollback. What I can do is raise it as a change today and get it into the next window, and here is a workaround for the meantime."

Q162. How do you handle a client asking for a status update every hour on a P1?

Answer: Do not treat it as an annoyance — it means they have their own stakeholders asking. Agree a fixed update cadence up front, for example every thirty minutes on the bridge, and hold to it. Predictable updates reduce the chasing, because the chasing is caused by uncertainty rather than impatience.

Q163. Why should the client trust someone with 13 months of experience with their systems?

Answer: Direct and confident: "Because of what I do rather than how long I have done it. I am already at L2, so the tickets I handle are the ones L1 could not solve. I do not act on a production system without a change record. I escalate with evidence rather than guessing. And I say when I do not know something, which is more useful to you than someone experienced who bluffs."

Q164. How would you handle two clients or departments both wanting priority?

Answer: Do not arbitrate it myself on a whim. Apply the impact and urgency matrix objectively, and if they genuinely tie, take it to my lead or the account manager, because relative business priority between two clients is a commercial decision and not mine. Be transparent with both about how the decision was made.

Q165. What business impact does your work have?

Answer: Show you understand the money, not just the ticket. "The reports I make sure load overnight are what the finance team closes the month on. The batch jobs I monitor are what moves transaction data between systems. When a Control-M job fails silently, somebody makes a decision on yesterday's data. My job is making sure that does not happen, and when it does, that it is caught before the business acts on it."

Tip: This is the strongest possible client-round answer. Prepare it properly.

Q166. What would you do if a client asked you to bypass the change process?

Answer: Say no, explain the risk in their terms — untested change, no rollback, potential outage that is worse than the original issue — and offer the correct route including the emergency change path if it genuinely qualifies. If they pushed, escalate to my lead rather than doing it. A client asking me to bypass process is a management conversation, not a decision for me.

Q167. How do you handle a knowledge transfer or handover to a client team?

Answer: Document rather than demonstrate. A live walkthrough is forgotten in a week. Written runbooks with the actual steps, common failures and escalation contacts, then a session to walk through them, then let them run it while I am available. The test is whether they can do it without me, not whether they understood while I was talking.

Q168. What if the client's own team caused the incident?

Answer: Fix it first, blame nobody during the incident. Afterwards, present the facts factually and without accusation — the timeline, what changed, what the logs show. Frame it as preventing recurrence rather than allocating fault. Being right about whose fault it was is worth very little; the relationship is worth more.

Q169. How would you communicate a major outage to non-technical stakeholders?

Answer: What is affected in business terms, who is affected, what is being done, when the next update is. Not the technical cause, which they cannot act on. "Order processing is unavailable for all users since 9:15. Technical teams are engaged and working on restoration. Next update at 10:00." Fixed cadence, plain language, no speculation on a cause you have not confirmed.

Q170. How do you gather requirements from a business user who cannot explain what they want?

Answer: Ask about the outcome rather than the solution. What are you trying to achieve, what do you do today, what goes wrong. Then get a concrete example — a specific record, a specific screen. Play it back in my own words to confirm before acting. Most unclear requirements are actually clear needs described in solution language.

Q171. Have you interacted with clients directly? What was your role?

Answer: Be honest about the level — you speak to business users on calls to collect details and explain fixes, and you have handled user escalations. Do not

claim client-manager-level exposure you do not have. Frame it accurately and it still reads well for a 1-year candidate.

Q172. How do you handle a client who does not accept your root cause finding?

Answer: Ask what they think the cause is, because they may know something about their environment that I do not. Then re-examine with that in mind. If the evidence still holds, present it more clearly rather than more forcefully — timeline, logs, the correlation. If the disagreement persists, involve L3 or my lead as a technical third opinion. Being believed matters more than being proven right, and evidence achieves that better than insistence.

Q173. What questions would you ask the client in your first week on the account?

Answer: What breaks most often and what has been most painful recently. Which systems the business cannot operate without. Who to contact for what, and their escalation preferences. What they think we get wrong today. That last one is the most valuable question and almost nobody asks it.

PART 11 — DIFFICULT AND TRICKY QUESTIONS (Q174–Q188)

Q174. Your resume looks written for this job description. Did you tailor it?

Answer: Own it confidently. "Yes. I read the description and made sure the things I actually do were described in the words you use for them. The content is my real work — I did not add anything I have not done. I would rather you see the relevant parts clearly than have to hunt for them."

Tip: Never look caught out by this. Tailoring is normal and expected. Denying it looks worse.

Q175. Everything on your resume is positive. What is not on there?

Answer: Give something real. "The tool depth is uneven, as I mentioned. I also have not worked across multiple clients, so my experience is deep on one account rather than broad. And I have not done ServiceNow development work beyond practice on my own instance."

Q176. If your current company offered you a counter-offer, would you stay?

Answer: "My reason for moving is scope, not money, so a salary counter would not change it. If they offered me the kind of application troubleshooting work I am looking for, I would consider it — I would be dishonest to say otherwise. But that is not what is available there, which is why I am here."

Tip: A flat "absolutely never" is not believable. The honest version is more convincing and shows the move is well reasoned.

Q177. What if we offer you 10 percent less than you asked?

Answer: Do not accept or reject instantly. "I would want to look at the full package rather than the fixed component alone. Can you tell me about the variable, the shift allowance and the review cycle? The role and the learning matter a lot to me, so I would rather find a workable number than lose the opportunity over it."

Q178. Rate yourself out of 10 on ServiceNow.

Answer: Give a number and justify the gap. "Six on ITSM operations, because I work incident, problem, change and CMDB daily and I am Administration Fundamentals certified. Three or four on development and scripting, because I have practised business rules and UI policies but I have not built anything in production. I gave myself six rather than eight because I have not done platform administration at scale or worked on integrations."

Tip: Never say 9 or 10. Never say 2. Justify the number by what you have *not* done — that is what makes the rating credible.

Q179. Rate yourself on SQL.

Answer: "Six or seven. I am comfortable with joins, aggregations, subqueries and window functions for investigation work, and I use PL/SQL on Oracle. Where I am weaker is performance tuning — I can spot a slow query but I would not claim to optimise an execution plan properly."

Q180. What if I told you your answer to the CMDB question was wrong?

Answer: Do not fold, and do not dig in. "Which part? I may have described how it works on my account rather than how it works generally, and those differ. I would rather understand where I have it wrong." This is a composure test — they may be testing whether you cave under pressure or defend without listening.

Q181. Why did you not get promoted at Cognizant?

Answer: "I have been there 13 months, and the cycle for a promotion is longer than that. What I did get was L2 ownership and the monitoring rotation earlier than the standard path, which is the substance of a step up even without the title change."

Q182. Your CSA is not complete. Why should we count it?

Answer: "You should not count it as complete, and I have written it as in progress for that reason. What I have completed is Administration Fundamentals, which is the prerequisite, last month. The CSA exam is scheduled for November. What I would ask you to count is the platform knowledge I use daily, not the certificate."

Q183. You listed AI and GitHub Copilot certifications. What have they got to do with support?

Answer: Turn it into a strength honestly. "Directly, not much, and I would not claim otherwise. Indirectly they are useful — I use AI tools to speed up log analysis patterns and to draft KB articles faster, and knowing their limits matters because I would not paste production data into an external tool. Mostly they show I keep learning outside what my role requires."

Q184. Ten tickets a day sounds low. Are you being underutilised?

Answer: "Ticket count is not my whole workload. I am also on the monitoring rotation and I own CMDB accuracy for my scope, and both take real time. If you measured me only on tickets it would look light, which is why I would point at the L2 mix and the non-ticket work. If anything I want more complexity, not more volume, which is part of why I am looking at this role."

Q185. What if this role turns out to be mostly L1 work?

Answer: Honest without being difficult. "I would do it properly — I have done L1 work and there is no snobbery about it. I would want to know the path to L2 work on the account, because that is where I add most value and where I want to grow. If the answer is that it is permanently L1 only, I would rather know now than in six months."

Tip: Do not say "I would refuse." Do not say "that is fine, I do not mind" either — that suggests no ambition.

Q186. Tell me something not on your resume.

Answer: Have something prepared that adds a dimension — a language, something you have taught yourself, the fact that you relocated from a small town in Shirpur to Pune for your first job. That last one is genuinely good, because it shows independence.

Q187. Sell me something you do not believe in.

Answer: Rare but it happens. Do not fake conviction. "I would not, in a client situation, because if I recommend something I do not believe in it comes back on me and on the account. What I would do is present it accurately with its limitations and let them decide, and if I actively thought it was wrong for them I would say so."

Q188. Do you think you are overqualified or underqualified for this role?

Answer: "Neither, and I think the 0 to 2 year band is honest about that. I am past needing to be taught what an incident is, and I am well short of knowing everything about application support. That gap is the reason I want the role."

PART 12 — PRIORITY LISTS

Your 48-hour revision plan. If you only have limited time, work top to bottom.

12.1 TOP 30 MOST IMPORTANT QUESTIONS

Prepare these to the point where you can deliver them without thinking.

#	Question	Where
1	Why are you leaving Cognizant after 13 months?	Danger 1
2	Tell me about yourself	2.1
3	Do you actually want support, or development?	Danger 2
4	Take me through your five tools — depth on each	Danger 3
5	Difference between incident, problem, request, change	Q22
6	Walk me through a typical day	Q1
7	Why should we hire you?	2.5
8	Why Accenture?	2.4
9	Tell me about a challenging piece of work (STAR)	Q33
10	What is your notice period and CTC?	Q114–115
11	What are your weaknesses?	Q112
12	What does Service Desk Management mean to you?	Q21
13	A ticket you escalated to L3 — what had you ruled out?	Q6
14	Tell me about a mistake you made	Q93
15	Tell me about a difficult user interaction	Q153
16	How is priority decided?	Q23
17	Why only 10 tickets a day?	Q2
18	Tell me about your CMDB work and why it matters	Q7
19	Are you comfortable with 24x7 rotational shifts?	Q118
20	Are you willing to relocate to Bengaluru?	Q117
21	How do you manage MCA with full-time work?	Danger 4
22	What are your strengths?	Q111
23	Where do you see yourself in 5 years?	2.6
24	Batch job failed at 2am — walk me through it	Q71
25	Describe a conflict you handled	Q94
26	Handling a tight deadline	Q95
27	Explain stakeholder management	Q96
28	What is your SLA target and what if you will breach?	Q3
29	Rate yourself on ServiceNow out of 10	Q178
30	Do you have any questions for us?	Q131

12.2 TOP 20 RESUME-BASED QUESTIONS

Q1, Q2, Q3, Q4, Q5, Q6, Q7, Q9, Q10, Q11, Q12, Q14, Q15, Q16, Q17, Q18, Q20, plus Danger 2, Danger 3 and Danger 4.

Focus: every one of these maps to a specific line on your resume. Read your resume line by line and ask yourself "what is the obvious question here." If you cannot answer it, either learn it or remove the line.

12.3 TOP 20 HR QUESTIONS

Q110 (why leaving), Q111 (strengths), Q112 (weaknesses), Q113 (salary), Q114 (CTC), Q115 (notice), Q116 (joining), Q117 (relocation), Q118 (shifts), Q119 (other interviews), Q120 (offers), Q122 (about Accenture), Q123 (why not someone senior), Q124 (criticism), Q127 (MCA), Q128 (different technology), Q130 (if not selected), Q131 (your questions), plus 2.4 (why Accenture) and 2.6 (five years).

Focus: stability and availability. Have your exact CTC, notice period and expected range written down before the call.

12.4 TOP 20 TECHNICAL QUESTIONS

Q22 (incident/problem/request/change), Q23 (priority), Q45 (ITIL), Q46 (CMDB), Q49 (incident vs request), Q50 (UI Policy vs Client Script), Q51 (UI Policy vs Data Policy), Q53 (Update Set), Q54 (ACL), Q55 (joins), Q57 (Linux commands), Q58 (duplicates query), Q59 (missing records query), Q64 (order of execution), Q66 (identification and reconciliation), Q67 (problem/known error/workaround), Q69 (AppDynamics slow app), Q71 (batch failure at 2am), Q73 (data mismatch investigation), Q85 (login failure).

Focus: Q50, Q51, Q53 and Q54 are the four ServiceNow questions most likely to be asked given your certification. Q58 and Q59 you should be able to write out by hand.

12.5 TOP 20 PROJECT-BASED QUESTIONS

Q33 (challenging work), Q34 (PDI), Q37 (college vs production), Q38 (root cause you found), Q39 (change/release work), Q40 (CMDB clean-up), Q41 (reporting automation), Q42 (proudest), Q43 (what you would do differently), Q44 (when you do not know), Q77 (recurring incident to closure), Q78 (reducing ticket volume), Q82 (tracing a transaction), Q86 (report with no data), Q88 (duplicate records from double run), Q92 (ticket reopened), Q98 (biggest achievement), Q104 (learning fast), Q146 (ownership beyond remit), plus Q35/Q36 if you re-add your college projects.

Focus: you need **three** STAR stories memorised cold. One technical investigation, one conflict or cross-team story, one deadline or pressure story. Almost every behavioural question in every round can be answered from those three with a different emphasis.

PART 13 — FINAL CHECKLIST

Before you apply

Get your exact notice period in writing from Cognizant HR
Know your exact current CTC — fixed, variable, total
Decide your expected range and your walk-away number
Confirm your MCA mode of study (distance / part-time / online)
Verify your Class X school name and percentage against your marksheet
Build one thing on your PDI, or remove that line from the resume
Find one real achievement from your Cognizant year

Your three STAR stories (write them out and rehearse)

1. **Technical investigation:** _____
2. **Conflict or cross-team:** _____
3. **Deadline or pressure:** _____

The four danger answers (rehearse out loud until natural)

Why leaving after 13 months
Support versus development ambition
Tool depth across your five platforms
MCA alongside full-time work

On the day

- Have your resume in front of you. They are reading from it, so you should be too.
- Numbers ready: 10 tickets a day, 95% SLA, 13 months, 9.52 CGPA, Aug 2026 certification, Nov 2026 CSA.
- Three questions to ask them.
- One recent Accenture fact, checked that week.

Three habits that will carry you

1. **Never bluff.** "I have not done that, but here is what I have done that is close" scores better than a confident wrong answer, and it is the only answer that survives follow-ups.
2. **Always give evidence.** A claim with a number or an example beats an adjective every time.
3. **Answer, then stop.** Over-talking is the most common way candidates undo a good answer. Say your piece and let them ask the next question.